# Numerical Analysis

Adam Kelly (ak2316@cam.ac.uk)

July 27, 2026

Almost none of the problems that we would like to solve can actually be solved. Most integrals have no closed form, most differential equations have no solution in terms of familiar functions, and a large linear system is not something one inverts by hand. Numerical analysis is the study of what to do about this: how to replace an infinite-dimensional problem by a finite-dimensional one whose answer we can compute, and – crucially – how to say something rigorous about the difference between the two.

So this is not a course about computers, and it is certainly not a course about programming. It is a course about *approximation*, and about the two questions that any approximation scheme must answer. How accurate is it? And how expensive is it? Almost every result in these notes is an answer to one of those two questions, and a surprising amount of quite beautiful classical mathematics – orthogonal polynomials, Rolle's theorem, the maximum modulus principle – turns out to be exactly what is needed to answer them.

This article constitutes my notes for the 'Numerical Analysis' course, held in Lent 2022 at Cambridge. These notes are *not a transcription of the lectures*, and differ significantly in quite a few areas. Still, all lectured material should be covered.

## Contents

**1 Polynomial Interpolation** **2**
- 1.1 Lagrange and Newton Polynomials . . . 2
- 1.2 Examples . . . 6
- 1.3 The Error of Interpolation . . . 8
- 1.4 The Runge Phenomenon . . . 9

**2 Orthogonal Polynomials** **10**
- 2.1 Inner Products on Function Spaces . . . 11
- 2.2 Orthogonal Polynomials . . . 12
- 2.3 The Three-Term Recurrence . . . 13
- 2.4 Least-Squares Approximation by Polynomials . . . 15
- 2.5 Least Squares with Discrete Data . . . 17

**3 Approximation of Linear Functionals** **17**
- 3.1 Newton–Cotes Formulae . . . 18
- 3.2 Gaussian Quadrature . . . 20

**4 Expressing Errors in Terms of Derivatives** **23**
- 4.1 The Peano Kernel Theorem . . . 24

4.2 Worked Kernels . . . . . 25

**5 Ordinary Differential Equations** **27**
5.1 One-Step Methods . . . . . 28
5.2 The Theta Method . . . . . 30
5.3 Multistep Methods . . . . . 31
5.4 The Root Condition and Dahlquist's Theorem . . . . . 33
5.5 Constructing Good Multistep Methods . . . . . 34
5.6 Runge–Kutta Methods . . . . . 36

**6 Stiff Equations and Stability** **38**
6.1 Linear Stability Domains . . . . . 39
6.2 Stability of Multistep and Runge–Kutta Methods . . . . . 41
6.3 Implementation . . . . . 42

**7 Numerical Linear Algebra** **44**
7.1 Triangular Systems . . . . . 44
7.2 LU Factorisation . . . . . 44
7.3 Pivoting . . . . . 46
7.4 Symmetric Matrices and Cholesky . . . . . 47
7.5 Banded and Sparse Matrices . . . . . 48

**8 QR Factorisation and Least Squares** **49**
8.1 Gram–Schmidt . . . . . 50
8.2 Givens Rotations . . . . . 51
8.3 Householder Reflections . . . . . 52
8.4 Linear Least Squares . . . . . 54

# §1 Polynomial Interpolation

Polynomials are the simplest functions we know how to compute with: to evaluate one we need only additions and multiplications, and to integrate or differentiate one we need only the power rule. It is therefore natural that our first move, when confronted with an awkward function $f$, is to replace it by a polynomial that agrees with $f$ at a handful of points. This is the *interpolation problem*, and it is the foundation on which almost everything else in this course is built – quadrature rules, multistep methods for differential equations and error estimates all come, in the end, from interpolating polynomials.

Throughout, we write $\mathcal{P}_n$ (or $\mathcal{P}_n[x]$ when we want to emphasise the variable) for the vector space of real polynomials of degree at most $n$. Note that $\dim \mathcal{P}_n = n+1$, since a polynomial in $\mathcal{P}_n$ is determined by its $n+1$ coefficients. That number, $n+1$, is worth keeping in mind: it is exactly the number of data points that we shall be able to fit.

## §1.1 Lagrange and Newton Polynomials

Let $f : [a,b] \to \mathbb{R}$ be a real-valued continuous function defined on some interval $[a,b]$ and let $(x_i)_{i=0}^n$ be $n+1$ distinct points in $[a,b]$. We wish to construct a polynomial $p$ of degree $n$ which interpolates $f$ at these points, i.e., satisfies

$$p(x_i) = f(x_i), \quad i = \overline{0,n}, \quad p \in \mathcal{P}_n$$

**Theorem 1.1** (Existence and Uniqueness)

Given $f \in C[a,b]$ and $n+1$ distinct points $(x_i)_{i=0}^n \in [a,b]$, there is exactly one polynomial $p \in \mathcal{P}_n$ such that $p(x_i) = f(x_i)$ for all $i$.

*Proof. Existence.* There is at least one polynomial interpolant $p \in \mathcal{P}_n$, the one in the Lagrange form,

$$p(x) = \sum_{i=0}^{n} f(x_i)\,\ell_i(x) \quad \text{with} \quad \ell_i(x) := \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j}, i = 0, \dots, n, \tag{†}$$

the $\ell_i$ s are called the **fundamental Lagrange polynomials**. Each $\ell_i$ is the product of $n$ linear factors, hence $\ell_i \in \mathcal{P}_n$. It also equals 1 at $x_i$ and vanishes at $x_j \neq x_i$, i.e., $\ell_i(x_j) = \delta_{ij}$. Therefore $p \in \mathcal{P}_n$ and

$$p(x_j) = \sum_{i=0}^{n} f(x_i)\,\ell_i(x_j) = f(x_j).$$

*Uniqueness.* There is at most one polynomial interpolant $p \in \mathcal{P}_n$ to $f$ on $(x_i)_{i=0}^n$. For if there are two, $p, q \in \mathcal{P}_n$, then the polynomial $r := p - q$ is of degree $n$ and vanishes at $n+1$ points, whence $r \equiv 0$. □

Let us introduce the so-called **nodal polynomial**

$$\omega(x) = \prod_{i=0}^{n} (x - x_i).$$

Then, in the expression (†) for $\ell_i$, the numerator is simply $\omega(x)/(x - x_i)$ while the denominator is equal to $w'(x_i)$. With that we arrive to a compact Lagrange form

$$\boxed{p(x) = \sum_{i=0}^{n} f(x_i)\,\ell_i(x) = \sum_{i=0}^{n} \frac{f(x_i)}{\omega'(x_i)} \frac{\omega(x)}{x - x_i}}.$$

The lagrange forms above and in (†) for interpolating polynomials are easy to manipulate, but they are unsuitable for numerical evaluation. An alternative is the *Newton form* which has an *adaptive* nature.

**Method 1.2** (Newton Form)

For $k = 0, 1, \dots, n$, let $p_k \in \mathcal{P}_k$ be the polynomial interpolant to $f$ on $x_0, \dots, x_k$. Then two subsequent $p_{k-1}$ and $p_k$ interpolate the same values $f(x_i)$ for $i \leq k - 1$, hence their difference is a polynomial of degree $k$ that vanishes at $k$ points $x_0, \dots, x_{k-1}$. Thus

$$p_k(x) - p_{k-1}(x) = A_k \prod_{i=0}^{k-1} (x - x_i), \tag{1}$$

with some constant $A_k$ which is seen to be equal to the leading coefficient of $p_k$.

It follows that $p = p_n$ can be built step by step as one constructs the sequence $(p_0, p_1, \dots)$, with $p_k$ obtained from $p_{k-1}$ by adding the term on the right-hand side

of (1), so that finally

$$p(x) = p_n(x) = p_0(x) + \sum_{k=1}^{n} [p_k(x) - p_{k-1}(x)] = \sum_{k=0}^{n} A_k \prod_{i=0}^{k-1} (x - x_i) .$$

The coefficients $A_k$ appearing here are important enough to be given a name of their own.

**Definition 1.3** (Divided Difference)

Given $f \in C[a, b]$ and $k + 1$ distinct points $(x_i)_{i=0}^{k} \in [a, b]$, the **divided difference** $f[x_0, \dots, x_k]$ of order $k$ is the leading coefficient of the polynomial $p_k \in \mathcal{P}_k$ which interpolates $f$ at these points.

The divided difference is a symmetric function of the variables $[x_0, \dots, x_k]$, since the interpolating polynomial does not care about the order in which we list the interpolation points. Also, if $f(x) = x^m$ with $m \leq k$, then $p_k = f$ and so $f[x_0, \dots, x_k] = \delta_{km}$. The two lowest-order cases are worth recording: $f[x_0] = f(x_0)$, and $f[x_0, x_1]$ is the slope

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

of the straight line through $(x_0, f(x_0))$ and $(x_1, f(x_1))$, which is where the name 'divided difference' comes from.

With this definition we arrive at the **Newton formula** for the interpolating polynomial.

**Theorem 1.4** (Newton Formula)

Given $n + 1$ distinct points $(x_i)_{i=0}^{n}$, let $p_n \in \mathcal{P}_n$ be the polynomial that interpolates $f$ at these points. Then it may be written in the Newton form

$$\begin{aligned} p_n(x) =& f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots \\ & \cdots + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \cdots (x - x_{n-1}) , \end{aligned}$$

or, more compactly,

$$p_n(x) = \sum_{k=0}^{n} f[x_0, \dots, x_k] \prod_{i=0}^{k-1} (x - x_i)$$

To make this formula of any use, we need an expression for $f[x_0, \dots, x_k]$. One such can be derived from the Lagrange formula by identifying the leading coefficient of $p$. This turns out to be

$$f[x_0, \dots, x_n] = \sum_{i=0}^{n} \frac{f(x_i)}{\omega'(x_i)}, \quad \omega(x) := \prod_{i=0}^{n} (x - x_i) .$$

However, this expression has computational disadvantages as the Lagrange form itself. A useful way to calculate divided difference is again an adaptive (or recurrence) approach.

**Theorem 1.5** (Recurrence Relation)

For distinct $x_0, x_1, \ldots, x_k$ with $k \geq 1$, we have

$$f[x_0, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}. \tag{2}$$

*Proof.* Let $q_0, q_1 \in \mathcal{P}_{k-1}$ be the polynomials such that $q_0$ interpolates $f$ on $(x_0, x_1, \ldots, x_{k-1})$ and $q_1$ interpolates on $(x_1, \ldots, x_k)$. Consider the polynomial

$$p(x) = \frac{x - x_0}{x_k - x_0} q_1(x) + \frac{x_k - x}{x_k - x_0} q_0(x), \quad p \in \mathcal{P}_k.$$

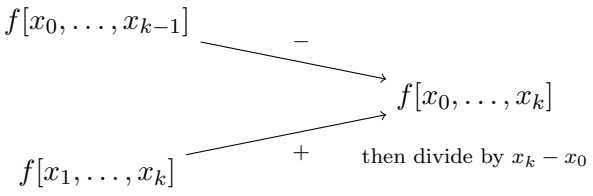
One readily sees that $p(x_i) = f(x_i)$ for all $i$: at $x_0$ the first term vanishes and the second has coefficient 1, at $x_k$ the reverse happens, and at an intermediate $x_i$ both $q_0$ and $q_1$ return $f(x_i)$ while the two coefficients sum to 1. Hence $p$ is the $k$-th degree interpolating polynomial to $f$. Moreover, the leading coefficient of $p$ is equal to the difference of those of $q_1$ and $q_0$ divided by $x_k - x_0$, and that is exactly what the recurrence (2) says. □

The recursive relation allows for the fast evaluation of the **divided difference table**

| $x_i$ | $f_i$ | $f[*,*]$ | $f[*,*,*]$ | $\cdots$ | $f[*,\cdots,*]$ |
|---|---|---|---|---|---|
| $x_0$ | $f[x_0]$ | | | | |
| | | $f[x_0,x_1]$ | | | |
| $x_1$ | $f[x_1]$ | | $f[x_0,x_1,x_2]$ | | |
| | | $f[x_1,x_2]$ | | $\ddots$ | |
| $x_2$ | $f[x_2]$ | | $f[x_2,x_3,x_4]$ | $\cdots$ | $f[x_0,x_1,\cdots,x_n]$ |
| | | $f[x_2,x_3]$ | | $\iddots$ | |
| $x_3$ | $f[x_3]$ | | $\iddots$ | | |
| $\vdots$ | $\vdots$ | $\iddots$ | | | |
| $x_n$ | $f[x_n]$ | | | | |

This can be done in $\mathcal{O}(n^2)$ operations and the outcome is the numbers $\{f[x_0, \ldots, x_k]\}_{k=0}^n$ at the head of the columns which can be used in the Newton form.

Pictorially, each entry is obtained from its two neighbours to the left, in the pattern




so that the table is filled column by column, each column being one entry shorter than the last.

Finally evaluation of $p$ at a given point $x$ using Newton formula (provided that divided differences $A_k = f[x_0, \ldots, x_k]$ are known) requires just $n$ multiplications, as long as we

do it by the **Horner scheme**

$$\begin{aligned} p_n(x) =& \{\cdots\{\{A_n \times (x - x_{n-1}) + A_{n-1}\} \times (x - x_{n-2}) + A_{n-2}\} \\ & \times (x - x_{n-3}) + \cdots + A_1\} \times (x - x_0) + A_0. \end{aligned}$$

It is worth pausing to compare the two forms. Both cost $\mathcal{O}(n^2)$ operations to set up. The Lagrange form is the one to reach for when we want to *manipulate* the interpolant symbolically – to integrate it, say, or to differentiate it – because $f$ appears in it linearly, through the values $f(x_i)$, with coefficients $\ell_i$ that do not depend on $f$ at all. This is exactly what we will exploit when we build quadrature rules in Section 3. The Newton form, on the other hand, is the one to reach for when we want to *compute*: adding a new interpolation point $x_{n+1}$ costs only one extra term, whereas in the Lagrange form every single $\ell_i$ has to be recomputed from scratch.

## §1.2 Examples

We will now look at some examples of these methods in use.

**Example 1.6** (Interpolating Polynomial)
Given the data

| $x_i$ | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| $f(x_i)$ | $-3$ | $-3$ | $-1$ | 9 |

we want to find the interpolating polynomial $p \in \mathcal{P}_3$ in both Lagrange and Newton forms.

*Fundamental Lagrange polynomials.* We have

$$\begin{aligned} \ell_0(x) &= \frac{(x-1)(x-2)(x-3)}{-6} = -\frac{1}{6}\left(x^3 - 6x^2 + 11x - 6\right), \\ \ell_1(x) &= \frac{x(x-2)(x-3)}{2} = \frac{1}{2}\left(x^3 - 5x^2 + 6x\right), \\ \ell_2(x) &= \frac{x(x-1)(x-3)}{-2} = -\frac{1}{2}\left(x^3 - 4x^2 + 3x\right), \\ \ell_3(x) &= \frac{x(x-1)(x-2)}{6} = \frac{1}{6}\left(x^3 - 3x^2 + 2x\right). \end{aligned}$$

*Lagrange form.* Using the polynomials above, we can write

$$\begin{aligned} p(x) &= (-3)\cdot\ell_0(x) + (-3)\cdot\ell_1(x) + (-1)\cdot\ell_2(x) + 9\cdot\ell_3(x) \\ &= \left(\frac{1}{2} - \frac{3}{2} + \frac{1}{2} + \frac{3}{2}\right)x^3 + \left(-3 + \frac{15}{2} - 2 - \frac{9}{2}\right)x^2 + \left(\frac{11}{2} - 9 + \frac{3}{2} + 3\right)x - 3 \\ &= x^3 - 2x^2 + x - 3. \end{aligned}$$

*Newton form.* Here we build the divided difference table, filling it in column by column using the recurrence (2). Since the nodes are spaced one apart, the denominators $x_k - x_0$ are $1, 2, 3$ in the three successive columns.

| $x_i$ | $f[x_i]$ | $f[*,*]$ | $f[*,*,*]$ | $f[*,*,*,*]$ |
|---|---|---|---|---|
| 0 | $-3$ | | | |
| | | 0 | | |
| 1 | $-3$ | | 1 | |
| | | 2 | | 1 |
| 2 | $-1$ | | 4 | |
| | | 10 | | |
| 3 | 9 | | | |

Reading off the numbers at the head of each column gives

$$p(x) = -3 + 0 \cdot (x-0) + 1 \cdot (x-0)(x-1) + 1 \cdot (x-0)(x-1)(x-2),$$

and expanding this out we do indeed recover $p(x) = x^3 - 2x^2 + x - 3$, as we must.

Notice how much less work the second computation was. The advantage becomes more pronounced still if the data arrives one point at a time.

**Example 1.7** (Adding a Point)

Suppose that to the data of the previous example we now append the point $x_4 = 4$, $f(x_4) = 33$. In the Lagrange form we would have to start again. In the Newton form we merely extend the table by one diagonal:

$$f[x_3, x_4] = \frac{33-9}{1} = 24, \quad f[x_2, x_3, x_4] = \frac{24-10}{2} = 7, \quad f[x_1, \dots, x_4] = \frac{7-4}{2} = \frac{3}{2},$$

and finally $f[x_0, \dots, x_4] = (\frac{3}{2} - 1)/4 = \frac{1}{8}$. Hence

$$p_4(x) = p_3(x) + \tfrac{1}{8}x(x-1)(x-2)(x-3).$$

All the work done for $p_3$ has been retained.

**Example 1.8** (A Divided Difference Table with Non-uniform Nodes)

Let $f(x) = \sqrt{x}$ and take the nodes $x_0 = 1$, $x_1 = 4$, $x_2 = 9$. Then

$$f[x_0, x_1] = \frac{2-1}{4-1} = \frac{1}{3}, \qquad f[x_1, x_2] = \frac{3-2}{9-4} = \frac{1}{5},$$

and so

$$f[x_0, x_1, x_2] = \frac{\frac{1}{5} - \frac{1}{3}}{9-1} = -\frac{1}{60}.$$

The interpolant is therefore

$$p_2(x) = 1 + \tfrac{1}{3}(x-1) - \tfrac{1}{60}(x-1)(x-4).$$

Evaluating at $x = 2$ gives $p_2(2) = 1 + \frac{1}{3} + \frac{1}{30} = 1.3\overline{6}$, against the true value $\sqrt{2} = 1.41421\ldots$. The error is about 0.048, which – as we are about to see – is entirely predictable.

## §1.3 The Error of Interpolation

An interpolant is only useful if we can say how far it is from $f$ *between* the interpolation points. The following theorem, which is the workhorse of the whole subject, says that the error looks exactly like the next term of a Taylor series, with the monomial $(x-a)^{n+1}$ replaced by the nodal polynomial $\omega$.

**Theorem 1.9** (Interpolation Error)

Let $f \in C^{n+1}[a,b]$ and let $x_0, \dots, x_n \in [a,b]$ be distinct. Let $p \in \mathcal{P}_n$ interpolate $f$ at these points. Then for every $x \in [a,b]$ there exists $\xi = \xi(x) \in (a,b)$ such that

$$f(x) - p(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi)\, \omega(x), \qquad \omega(x) = \prod_{i=0}^{n} (x - x_i).$$

*Proof.* If $x = x_i$ for some $i$ then both sides vanish and there is nothing to prove, so fix $x \in [a,b]$ with $x \neq x_i$ for all $i$. In particular $\omega(x) \neq 0$, so we may define $\varphi : [a,b] \to \mathbb{R}$ by

$$\varphi(t) = f(t) - p(t) - \big(f(x) - p(x)\big)\frac{\omega(t)}{\omega(x)}.$$

This is a $C^{n+1}$ function of $t$, and it has been rigged so that $\varphi(x) = 0$; it also vanishes at each of $x_0, \dots, x_n$, since there $f - p$ and $\omega$ both vanish. So $\varphi$ has at least $n+2$ distinct zeros in $[a,b]$.

Now we apply Rolle's theorem repeatedly. Between consecutive zeros of $\varphi$ there is a zero of $\varphi'$, so $\varphi'$ has at least $n+1$ distinct zeros; between consecutive zeros of $\varphi'$ there is a zero of $\varphi''$, so $\varphi''$ has at least $n$ distinct zeros; and continuing in this way, $\varphi^{(n+1)}$ has at least one zero, say $\xi \in (a,b)$.

But $p \in \mathcal{P}_n$ has $p^{(n+1)} \equiv 0$, while $\omega$ is monic of degree $n+1$ and so $\omega^{(n+1)} \equiv (n+1)!$. Hence

$$0 = \varphi^{(n+1)}(\xi) = f^{(n+1)}(\xi) - \big(f(x) - p(x)\big)\frac{(n+1)!}{\omega(x)},$$

and rearranging gives the result. □

The same trick – count zeros, then apply Rolle – gives a very satisfying interpretation of divided differences: they are, up to a factorial, derivatives.

**Theorem 1.10** (Divided Differences and Derivatives)

Let $x_0, \dots, x_n$ be distinct points and let $[a,b]$ be the smallest interval containing them. If $f \in C^n[a,b]$, then there is some $\xi \in (a,b)$ with

$$f[x_0, \dots, x_n] = \frac{1}{n!} f^{(n)}(\xi).$$

*Proof.* Let $p \in \mathcal{P}_n$ interpolate $f$ at the $n+1$ points. Then $f - p$ vanishes at $n+1$ distinct points, so by Rolle applied $n$ times, $(f-p)^{(n)}$ vanishes somewhere in $(a,b)$, say at $\xi$. But $p$ has leading coefficient $f[x_0, \dots, x_n]$, so $p^{(n)} \equiv n!\, f[x_0, \dots, x_n]$, and thus

$$0 = f^{(n)}(\xi) - n!\, f[x_0, \dots, x_n].$$ □

**Remark.** This is a pleasing statement in its own right: it says that the divided difference is a 'discrete derivative', and it also lets us make sense of $f[x_0, \dots, x_n]$ when some of the points coalesce. Letting all the $x_i \to x$, the natural definition is $f[x, x, \dots, x] = f^{(n)}(x)/n!$, and with this convention the Newton formula becomes the Taylor expansion of $f$ about $x$. Interpolation and Taylor expansion are, in this sense, the same thing.

Let us use Theorem 1.9 on the example above.

**Example 1.11**

With $f(x) = \sqrt{x}$ and nodes $1, 4, 9$, we have $f'''(x) = \frac{3}{8}x^{-5/2}$ and $\omega(2) = (2-1)(2-4)(2-9) = 14$. So

$$\sqrt{2} - p_2(2) = \frac{1}{6} \cdot \frac{3}{8}\xi^{-5/2} \cdot 14 = \frac{7}{8}\xi^{-5/2}$$

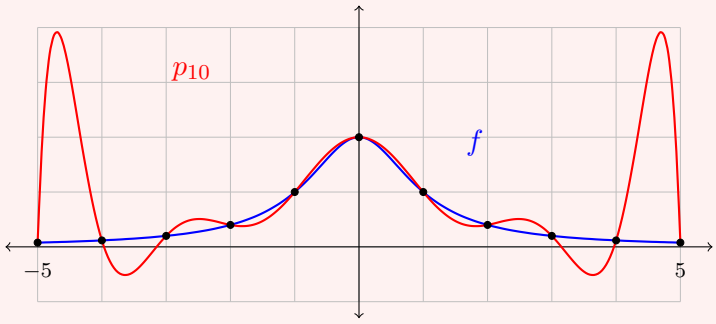
for some $\xi \in (1, 9)$. Since $\xi > 1$ we get $\left|\sqrt{2} - p_2(2)\right| \leq 7/8$, which is true but useless; a better bound comes from noticing that $\xi$ is in practice near the interval where the action is. The exact error was $0.0476$, corresponding to $\xi \approx 2.2$. The moral is that the theorem gives the *form* of the error very precisely, but a crude bound on $f^{(n+1)}$ can throw most of that precision away.

## §1.4 The Runge Phenomenon

Theorem 1.9 contains two factors that we might hope to control. The factor $f^{(n+1)}(\xi)/(n+1)!$ is out of our hands – it is a property of $f$. But the nodal polynomial $\omega$ depends only on where we choose to put our interpolation points, and this we *can* control. The following famous example of Runge shows that the choice matters enormously, and that the obvious choice is a bad one.

**Example 1.12** (Runge's Example)

Let $f(x) = (1 + x^2)^{-1}$ on $[-5, 5]$, and interpolate at the $n+1$ equally spaced points $x_j = -5 + 10j/n$. Below is the case $n = 10$: the function $f$ is in blue, the interpolant $p_{10}$ in red, with the interpolation points marked.




The interpolant is faithful in the middle of the interval and catastrophically bad near the endpoints, where it oscillates with amplitude close to 2 even though $|f| \leq 1$ everywhere. Worse, this does not improve as $n$ grows: one can show that

$$\max_{x \in [-5,5]} |f(x) - p_n(x)| \to \infty \quad \text{as } n \to \infty.$$

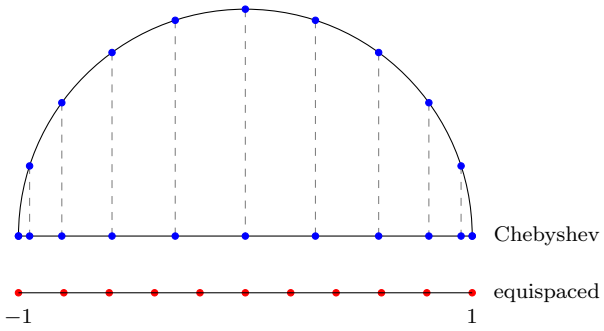
Interpolating at more equally spaced points makes matters *worse*, not better.

What has gone wrong? The culprit is the nodal polynomial. For equally spaced nodes on $[-1,1]$, $|\omega|$ is tiny in the middle of the interval but grows by a factor exponential in $n$ as we approach the ends, so the error bound is wildly non-uniform. If we want $|\omega|$ to be uniformly small, we should not space the nodes uniformly; we should cluster them near the endpoints.

The optimal choice is classical. On $[-1,1]$ the **Chebyshev points**

$$x_j = \cos\left(\frac{(n-j)\pi}{n}\right), \qquad j = 0, 1, \dots, n,$$

are obtained by taking $n+1$ equally spaced points on the upper half of the unit circle and projecting them down onto the real axis. The picture makes the clustering obvious.




With this choice one can show that the nodal polynomial satisfies

$$\max_{x\in[-1,1]} |\omega(x)| = 2^{-n},$$

and that no other choice of $n+1$ nodes does better. (The nodal polynomial for the Chebyshev points is $2^{-n}T_{n+1}$, where $T_{n+1}$ is the Chebyshev polynomial we shall meet in the next section; the optimality is a consequence of its equioscillation.) For a general interval $[a,b]$ we simply rescale, taking

$$x_j = \frac{a+b}{2} + \frac{b-a}{2}\cos\left(\frac{(n-j)\pi}{n}\right).$$

Repeating Runge's experiment with Chebyshev points, the interpolant converges uniformly to $f$, and rapidly at that.

**Remark.** This is a good point to pause and notice what the theory has bought us. Nothing about the function $f$ has changed; all we have done is move the sampling points. But the same theorem that predicted disaster for equispaced nodes predicts success for Chebyshev nodes, because it isolates precisely the quantity $(\omega)$ that we are free to choose. This pattern – write down an exact error formula, look at which factor is under our control, and optimise it – recurs throughout the course, and is the whole idea behind Gaussian quadrature.

## §2 Orthogonal Polynomials

We now change tack somewhat. Rather than asking for a polynomial that agrees with $f$ at finitely many points, we ask for a polynomial that is close to $f$ *on average*, in the sense

of least squares. This turns out to require a good basis for $\mathcal{P}_n$, and the right notion of 'good' is orthogonality with respect to a suitable inner product. Orthogonal polynomials are the central objects here, and they will reappear – somewhat unexpectedly – as the key to the most accurate quadrature rules there are.

## §2.1 Inner Products on Function Spaces

**Definition 2.1** (Inner Product)

Let $V$ be a real vector space. An **inner product** (or scalar product) on $V$ is a map $\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$ which is

(i) *symmetric*: $\langle f, g \rangle = \langle g, f \rangle$ for all $f, g \in V$;

(ii) *bilinear*: linear in each argument;

(iii) *positive definite*: $\langle f, f \rangle \geq 0$ for all $f$, with equality if and only if $f = 0$.

We say $f$ and $g$ are **orthogonal** if $\langle f, g \rangle = 0$, and we write $\|f\| = \langle f, f \rangle^{1/2}$ for the induced norm.

For us $V$ will almost always be a space of functions on an interval, and the inner product will be of the following shape.

**Definition 2.2** (Weight Function)

Let $[a, b] \subseteq \mathbb{R}$ (possibly infinite) and let $w : (a, b) \to \mathbb{R}$ be continuous, non-negative, and non-zero except at isolated points, with $\int_a^b w(x)\,\mathrm{d}x < \infty$. We call $w$ a **weight function**, and define

$$\langle f, g \rangle = \int_a^b w(x) f(x) g(x)\,\mathrm{d}x.$$

The conditions on $w$ are exactly what is needed for this to be an inner product on $C[a, b]$: symmetry and bilinearity are immediate, and if $\langle f, f \rangle = \int_a^b w f^2 = 0$ with $w f^2 \geq 0$ continuous, then $w f^2 \equiv 0$, and since $w$ vanishes only at isolated points, $f \equiv 0$ by continuity.

A second, more down-to-earth example is available when we only know $f$ at finitely many points.

**Example 2.3** (Discrete Inner Product)

Let $\xi_1, \ldots, \xi_m$ be distinct points and set

$$\langle f, g \rangle = \sum_{j=1}^{m} f(\xi_j) g(\xi_j).$$

This is symmetric and bilinear, and $\langle f, f \rangle = \sum_j f(\xi_j)^2 \geq 0$. It is *not* positive definite on all of $C[a, b]$ – a function vanishing at every $\xi_j$ has zero norm – but it *is* positive definite on $\mathcal{P}_n$ provided $n < m$, since a nonzero polynomial of degree at most $n$ cannot vanish at $m > n$ points. This is the inner product relevant to fitting a curve to experimental data.

## §2.2 Orthogonal Polynomials

**Definition 2.4** (Orthogonal Polynomials)

A sequence $p_0, p_1, p_2, \dots$ of polynomials is a sequence of **orthogonal polynomials** with respect to $\langle \cdot, \cdot \rangle$ if $\deg p_n = n$ for every $n$, and $\langle p_n, p_m \rangle = 0$ whenever $n \neq m$.

Since $\deg p_n = n$ exactly, the polynomials $p_0, \dots, p_n$ are automatically linearly independent (they have distinct degrees), and so they form an orthogonal *basis* of $\mathcal{P}_n$. This is really the point of the definition: we are constructing a basis of $\mathcal{P}_n$ in which computations decouple.

Of course, orthogonal polynomials are only determined up to scaling, so to speak of *the* orthogonal polynomials we should normalise. The usual convention in this course is to take them **monic**, that is, with leading coefficient 1.

**Theorem 2.5** (Existence and Uniqueness)

For any inner product on the space of polynomials there exists a unique sequence of monic orthogonal polynomials $p_0, p_1, \dots$, and these may be constructed by the Gram–Schmidt formula

$$p_{n+1}(x) = x^{n+1} - \sum_{k=0}^{n} \frac{\langle x^{n+1}, p_k \rangle}{\langle p_k, p_k \rangle} p_k(x).$$

*Proof.* We induct on $n$. For $n = 0$ the only monic polynomial of degree 0 is $p_0 \equiv 1$, so existence and uniqueness are clear. Suppose then that $p_0, \dots, p_n$ have been found. Define $p_{n+1}$ by the displayed formula. The subtracted sum lies in $\mathcal{P}_n$, so $p_{n+1}$ is monic of degree $n+1$. For each $j \leq n$, using orthogonality of the $p_k$,

$$\langle p_{n+1}, p_j \rangle = \langle x^{n+1}, p_j \rangle - \frac{\langle x^{n+1}, p_j \rangle}{\langle p_j, p_j \rangle} \langle p_j, p_j \rangle = 0,$$

so $p_{n+1}$ is orthogonal to $p_0, \dots, p_n$ and hence to all of $\mathcal{P}_n$.

For uniqueness, suppose $\tilde{p}_{n+1}$ were another monic polynomial of degree $n+1$ orthogonal to $\mathcal{P}_n$. Then $q = p_{n+1} - \tilde{p}_{n+1}$ has degree at most $n$, so $q \in \mathcal{P}_n$, and $q$ is orthogonal to everything in $\mathcal{P}_n$ – in particular to itself. Hence $\langle q, q \rangle = 0$ and so $q = 0$. □

**Remark.** Note the characterisation buried in that proof: $p_n$ is the unique monic polynomial of degree $n$ orthogonal to *every* polynomial of lower degree. We will use this form of the statement constantly.

Gram–Schmidt is a perfectly good proof, but it is a poor algorithm: computing $p_{n+1}$ requires all of $p_0, \dots, p_n$, so the cost grows and rounding errors accumulate. For the inner products we care about there is a far better recurrence, which involves only the previous *two* polynomials.

## §2.3 The Three-Term Recurrence

The key hypothesis is that multiplication by $x$ is self-adjoint. For the weighted-integral inner product this is obvious:

$$\langle xf, g\rangle = \int_a^b w(x)\, xf(x)g(x)\, \mathrm{d}x = \langle f, xg\rangle\,,$$

and it holds for the discrete inner product too.

**Theorem 2.6** (Three-Term Recurrence)

Suppose the inner product satisfies $\langle xf, g\rangle = \langle f, xg\rangle$ for all polynomials $f, g$. Then the monic orthogonal polynomials satisfy

$$p_{n+1}(x) = (x - \alpha_n)p_n(x) - \beta_n p_{n-1}(x), \qquad n \geq 0,$$

with the conventions $p_{-1} \equiv 0$, $p_0 \equiv 1$, $\beta_0$ arbitrary, and

$$\alpha_n = \frac{\langle p_n, xp_n\rangle}{\langle p_n, p_n\rangle}, \qquad \beta_n = \frac{\langle p_n, p_n\rangle}{\langle p_{n-1}, p_{n-1}\rangle} > 0 \quad (n \geq 1).$$

*Proof.* Both $p_{n+1}$ and $xp_n$ are monic of degree $n+1$, so their difference lies in $\mathcal{P}_n$, and since $p_0, \ldots, p_n$ is a basis of $\mathcal{P}_n$ we may write

$$p_{n+1}(x) - xp_n(x) = \sum_{k=0}^{n} \gamma_k p_k(x)$$

for some scalars $\gamma_k$. Taking the inner product with $p_j$ for $j \leq n$ and using orthogonality of $p_{n+1}$ against $\mathcal{P}_n$,

$$\gamma_j \langle p_j, p_j\rangle = -\langle xp_n, p_j\rangle = -\langle p_n, xp_j\rangle\,,$$

where the last step is the self-adjointness hypothesis. Now if $j \leq n-2$ then $xp_j \in \mathcal{P}_{n-1}$, which is orthogonal to $p_n$, so $\gamma_j = 0$. Only $j = n$ and $j = n-1$ survive, giving

$$p_{n+1}(x) = (x - \alpha_n)p_n(x) - \beta_n p_{n-1}(x), \quad \alpha_n = \frac{\langle p_n, xp_n\rangle}{\langle p_n, p_n\rangle}, \quad \beta_n = \frac{\langle p_n, xp_{n-1}\rangle}{\langle p_{n-1}, p_{n-1}\rangle}.$$

Finally, $xp_{n-1}$ is monic of degree $n$, so $xp_{n-1} - p_n \in \mathcal{P}_{n-1}$ is orthogonal to $p_n$; hence $\langle p_n, xp_{n-1}\rangle = \langle p_n, p_n\rangle$, which gives the stated $\beta_n$, manifestly positive. □

This is a genuinely useful algorithm: each new polynomial costs $\mathcal{O}(1)$ inner products rather than $\mathcal{O}(n)$, and the recurrence is numerically stable. Let's look at the two examples that matter most.

**Example 2.7** (Legendre Polynomials)

Take $[a, b] = [-1, 1]$ and $w \equiv 1$, so that $\langle f, g\rangle = \int_{-1}^{1} fg\, \mathrm{d}x$. The resulting monic orthogonal polynomials are the (monic) **Legendre polynomials**. By symmetry $\langle p_n, xp_n\rangle = 0$ for every $n$ – the integrand is odd – so $\alpha_n = 0$ throughout, and the

recurrence collapses to $p_{n+1} = xp_n - \beta_n p_{n-1}$. Computing,

$$p_0 = 1, \quad p_1 = x, \quad p_2 = x^2 - \tfrac{1}{3}, \quad p_3 = x^3 - \tfrac{3}{5}x, \quad p_4 = x^4 - \tfrac{6}{7}x^2 + \tfrac{3}{35}.$$

For instance $\beta_1 = \langle p_1, p_1\rangle / \langle p_0, p_0\rangle = (2/3)/2 = 1/3$, giving $p_2 = x \cdot x - \frac{1}{3} = x^2 - \frac{1}{3}$ as claimed. In the literature one usually meets these normalised so that $P_n(1) = 1$; then $P_0 = 1$, $P_1 = x$, $P_2 = \frac{1}{2}(3x^2 - 1)$, and so on.

**Example 2.8** (Chebyshev Polynomials)

Take $[a, b] = [-1, 1]$ with the weight $w(x) = (1 - x^2)^{-1/2}$, so

$$\langle f, g\rangle = \int_{-1}^{1} \frac{f(x)g(x)}{\sqrt{1 - x^2}}\, \mathrm{d}x.$$

(The weight blows up at $\pm 1$, but integrably, so this is fine.) Define $T_n$ by

$$T_n(\cos\theta) = \cos(n\theta).$$

That this really is a polynomial in $x = \cos\theta$ follows by induction from the identity $\cos((n+1)\theta) + \cos((n-1)\theta) = 2\cos\theta\cos(n\theta)$, which gives at once the recurrence

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \qquad T_0 = 1,\ T_1 = x.$$

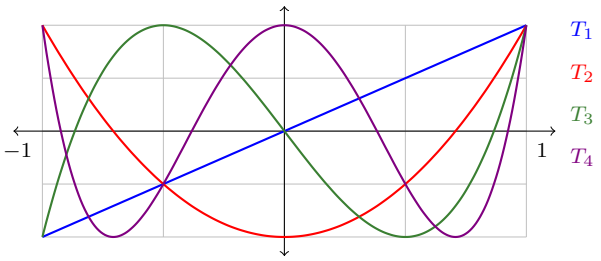
So $T_2 = 2x^2 - 1$, $T_3 = 4x^3 - 3x$, $T_4 = 8x^4 - 8x^2 + 1$, and in general $T_n$ has leading coefficient $2^{n-1}$ for $n \geq 1$.

To see orthogonality, substitute $x = \cos\theta$, $\mathrm{d}x = -\sin\theta\, \mathrm{d}\theta$, $\sqrt{1 - x^2} = \sin\theta$:

$$\langle T_n, T_m\rangle = \int_{-1}^{1} \frac{T_n T_m}{\sqrt{1 - x^2}}\, \mathrm{d}x = \int_0^{\pi} \cos(n\theta)\cos(m\theta)\, \mathrm{d}\theta = 0$$

for $n \neq m$, by the usual orthogonality of cosines. The monic Chebyshev polynomials are $2^{1-n}T_n$ for $n \geq 1$.

The Chebyshev polynomials are worth a picture, because their defining feature is visible at a glance: on $[-1, 1]$ they oscillate between $-1$ and $+1$, attaining those extreme values $n + 1$ times.




This *equioscillation* is precisely why the Chebyshev points of the previous section are optimal interpolation nodes: among all monic polynomials of degree $n$, the monic Chebyshev polynomial $2^{1-n}T_n$ is the one with the smallest maximum modulus on $[-1, 1]$, namely $2^{1-n}$. Any other monic $q$ of degree $n$ with $\|q\|_\infty < 2^{1-n}$ would have $2^{1-n}T_n - q$ alternating in sign at the $n + 1$ extrema of $T_n$, hence having $n$ zeros – impossible for a polynomial of degree $n - 1$ that is not identically zero.

There are other classical families, each attached to its own weight, which we mention for completeness:

| Family | Interval | Weight $w(x)$ |
|---|---|---|
| Legendre | $[-1,1]$ | $1$ |
| Chebyshev | $[-1,1]$ | $(1-x^2)^{-1/2}$ |
| Laguerre | $[0,\infty)$ | $e^{-x}$ |
| Hermite | $(-\infty,\infty)$ | $e^{-x^2}$ |

## §2.4 Least-Squares Approximation by Polynomials

We can now solve the approximation problem we set out with. Given $f$ and an inner product, we seek the $p \in \mathcal{P}_n$ minimising $\|f-p\|$. Since $\mathcal{P}_n$ is a finite-dimensional subspace, the answer is of course the orthogonal projection of $f$ onto it – and having an orthogonal basis to hand makes that projection completely explicit.

**Theorem 2.9** (Least-Squares Approximation)

Let $p_0, p_1, \dots$ be the orthogonal polynomials for $\langle\cdot,\cdot\rangle$. Then the unique minimiser of $\|f-p\|$ over $p \in \mathcal{P}_n$ is

$$\hat{p}_n = \sum_{k=0}^{n} \frac{\langle f, p_k\rangle}{\langle p_k, p_k\rangle} p_k.$$

*Proof.* Any $p \in \mathcal{P}_n$ may be written $p = \sum_{k=0}^{n} c_k p_k$. Expanding and using orthogonality,

$$\begin{aligned}\|f-p\|^2 &= \langle f - \textstyle\sum_k c_k p_k,\, f - \sum_k c_k p_k\rangle \\ &= \langle f, f\rangle - 2\sum_{k=0}^{n} c_k \langle f, p_k\rangle + \sum_{k=0}^{n} c_k^2 \langle p_k, p_k\rangle .\end{aligned}$$

The cross terms $\langle p_j, p_k\rangle$ with $j \neq k$ have vanished, so this is a sum of $n+1$ independent quadratics in the $c_k$. Completing the square in each,

$$\|f-p\|^2 = \langle f, f\rangle - \sum_{k=0}^{n} \frac{\langle f, p_k\rangle^2}{\langle p_k, p_k\rangle} + \sum_{k=0}^{n} \langle p_k, p_k\rangle \left(c_k - \frac{\langle f, p_k\rangle}{\langle p_k, p_k\rangle}\right)^2 .$$

Since $\langle p_k, p_k\rangle > 0$, this is minimised precisely when every bracket vanishes, i.e. $c_k = \langle f, p_k\rangle \,/\, \langle p_k, p_k\rangle$, and the minimiser is unique. □

Notice the structural virtue of this answer: the coefficient of $p_k$ does not depend on $n$. If we decide we want a better approximation, we simply compute one more coefficient and append one more term; nothing already computed is wasted. This is exactly the adaptivity we admired in the Newton form.

Reading off the value at the minimum in the proof gives the error, and this is worth recording separately.

**Corollary 2.10** (Bessel's Inequality and Pythagoras)

With $\hat{p}_n$ as above,

$$\|f - \hat{p}_n\|^2 = \langle f, f\rangle - \sum_{k=0}^{n} \frac{\langle f, p_k\rangle^2}{\langle p_k, p_k\rangle} = \|f\|^2 - \|\hat{p}_n\|^2 .$$

In particular $\sum_{k=0}^{n} \langle f, p_k\rangle^2 / \langle p_k, p_k\rangle \leq \|f\|^2$, and $\langle f - \hat{p}_n, \hat{p}_n\rangle = 0$.

That last statement – the error is orthogonal to the approximation – is Pythagoras' theorem, and it is the geometric content of the whole business. It also gives an alternative one-line proof of the theorem: for any $p \in \mathcal{P}_n$ we have $f - p = (f - \hat{p}_n) + (\hat{p}_n - p)$ with the two summands orthogonal, so $\|f - p\|^2 = \|f - \hat{p}_n\|^2 + \|\hat{p}_n - p\|^2 \geq \|f - \hat{p}_n\|^2$.

Since $\|f - \hat{p}_n\|$ is decreasing in $n$, it converges; the natural question is whether it converges to zero. It does, and the reason is a theorem from outside numerical analysis entirely.

**Theorem 2.11** (Weierstrass Approximation Theorem)

Let $f \in C[a, b]$ with $[a, b]$ finite. For every $\varepsilon > 0$ there is a polynomial $q$ with $|f(x) - q(x)| < \varepsilon$ for all $x \in [a, b]$.

We shall not prove this here – it is a piece of analysis rather than numerical analysis, and the reader will meet it elsewhere. What we want is the consequence.

**Corollary 2.12** (Convergence of Least-Squares Approximation)

Let $\langle f, g\rangle = \int_a^b wfg \, \mathrm{d}x$ with $[a, b]$ finite and $w$ a weight function. Then for $f \in C[a, b]$ we have $\|f - \hat{p}_n\| \to 0$ as $n \to \infty$.

*Proof.* Let $\varepsilon > 0$ and set $W = \int_a^b w(x) \, \mathrm{d}x < \infty$. By Weierstrass there is a polynomial $q$, of degree $m$ say, with $|f - q| < \varepsilon$ pointwise on $[a, b]$. Then

$$\|f - q\|^2 = \int_a^b w(x)(f(x) - q(x))^2 \, \mathrm{d}x \leq \varepsilon^2 W.$$

For $n \geq m$ we have $q \in \mathcal{P}_n$, so by minimality $\|f - \hat{p}_n\| \leq \|f - q\| \leq \varepsilon\sqrt{W}$. As $\varepsilon$ was arbitrary and $\|f - \hat{p}_n\|$ is non-increasing, the result follows. □

**Corollary 2.13** (Parseval's Identity)

Under the hypotheses above,

$$\sum_{k=0}^{\infty} \frac{\langle f, p_k\rangle^2}{\langle p_k, p_k\rangle} = \langle f, f\rangle .$$

*Proof.* Immediate from $\|f\|^2 - \sum_{k=0}^{n} \langle f, p_k\rangle^2 / \langle p_k, p_k\rangle = \|f - \hat{p}_n\|^2 \to 0$. □

**Example 2.14** (Least Squares Fit of $|x|$)

Take $[a,b] = [-1,1]$, $w \equiv 1$, and $f(x) = |x|$, and let us find the best quadratic approximation. Using the monic Legendre polynomials $p_0 = 1$, $p_1 = x$, $p_2 = x^2 - \frac{1}{3}$, we compute

$$\langle f, p_0 \rangle = \int_{-1}^{1} |x| \, \mathrm{d}x = 1, \qquad \langle f, p_1 \rangle = \int_{-1}^{1} |x| \, x \, \mathrm{d}x = 0,$$

$$\langle f, p_2 \rangle = \int_{-1}^{1} |x| \left(x^2 - \tfrac{1}{3}\right) \mathrm{d}x = 2 \int_{0}^{1} \left(x^3 - \tfrac{x}{3}\right) \mathrm{d}x = 2\left(\tfrac{1}{4} - \tfrac{1}{6}\right) = \tfrac{1}{6}.$$

Also $\langle p_0, p_0 \rangle = 2$ and $\langle p_2, p_2 \rangle = \int_{-1}^{1} (x^2 - \frac{1}{3})^2 \, \mathrm{d}x = \frac{8}{45}$. Hence

$$\hat{p}_2(x) = \tfrac{1}{2} + \tfrac{1/6}{8/45}\left(x^2 - \tfrac{1}{3}\right) = \tfrac{15}{16}x^2 + \tfrac{3}{16}.$$

A quick sanity check: $\hat{p}_2(\pm 1) = 18/16 > 1$ and $\hat{p}_2(0) = 3/16 > 0$, so the fit overshoots at the ends and at the kink, which is what one expects of a smooth curve chasing a corner.

### §2.5 Least Squares with Discrete Data

In practice one rarely knows $f$ everywhere; one knows $m$ measurements $f(\xi_1), \dots, f(\xi_m)$ at distinct points and wants the $p \in \mathcal{P}_n$ minimising

$$\sum_{j=1}^{m} \big(f(\xi_j) - p(\xi_j)\big)^2.$$

This is precisely the least-squares problem for the discrete inner product

$$\langle g, h \rangle = \sum_{j=1}^{m} g(\xi_j) h(\xi_j),$$

which as we noted is a genuine inner product on $\mathcal{P}_n$ as long as $n < m$. Everything above goes through verbatim: we build the orthogonal polynomials $p_0, \dots, p_n$ for this inner product using the three-term recurrence, which applies because multiplication by $x$ is again self-adjoint, and then Theorem 2.9 hands us the answer.

**Remark.** The naive alternative is to write $p = \sum_k c_k x^k$ and solve the resulting $(n+1) \times (n+1)$ linear system for the $c_k$ – the so-called *normal equations*. This works, but the matrix involved is close to the notorious Hilbert matrix and is horribly ill-conditioned even for modest $n$. Orthogonalising first turns the linear system into a diagonal one, and the improvement in accuracy is dramatic. We will meet the same phenomenon again, from a different direction, when we discuss the QR factorisation in Section 8.

## §3 Approximation of Linear Functionals

Almost every quantity we might want to extract from a function – its value at a point, its derivative there, its integral over an interval – is a *linear* function of $f$. This section is about approximating such quantities using only the simplest data of all, namely point values of $f$.

**Definition 3.1** (Linear Functional)

A **linear functional** on a real vector space $V$ of functions is a linear map $L : V \to \mathbb{R}$.

We usually do not fuss over the space $V$; we write down a formula for $L$ and take $V$ to be whatever space of functions makes the formula sensible.

**Example 3.2**

The following are all linear functionals.

(i) $L(f) = f(\xi)$ for some fixed $\xi$, on $V = C[a, b]$.

(ii) $L(f) = f'(\eta)$ for some fixed $\eta$, on $V = C^1[a, b]$.

(iii) $L(f) = \int_a^b w(x)f(x)\,\mathrm{d}x$, on $V = C[a, b]$.

(iv) Any linear combination of the above, for instance

$$L(f) = f(\beta) - f(\alpha) - \frac{\beta - \alpha}{2}\big(f'(\beta) + f'(\alpha)\big).$$

Our aim is to approximate a complicated $L$ (typically an integral) by a finite combination of simple ones,

$$L(f) \approx \sum_{i=0}^{N} a_i f(x_i),$$

where $x_0, \dots, x_N \in [a, b]$ are distinct. How should we choose the $a_i$, and the $x_i$?

The natural idea is one we already have: replace $f$ by its interpolating polynomial and apply $L$ to that. Using the fundamental Lagrange polynomials for the nodes $x_i$,

$$f(x) \approx \sum_{i=0}^{N} f(x_i)\ell_i(x) \quad \Longrightarrow \quad L(f) \approx \sum_{i=0}^{N} L(\ell_i)f(x_i),$$

so we should take $a_i = L(\ell_i)$. With this choice the approximation is *exact* for $f \in \mathcal{P}_N$, since then the interpolant is $f$ itself.

That is $N+1$ free parameters buying exactness on an $(N+1)$-dimensional space, which sounds like a fair deal. But we have another $N+1$ parameters that we have not used at all – the nodes $x_i$. If we choose those cleverly too, we might hope for exactness on $\mathcal{P}_{2N+1}$. Remarkably, for integrals we can achieve exactly that. This is Gaussian quadrature, and it is the subject of the rest of this section.

## §3.1 Newton–Cotes Formulae

First, though, let us see what the naive choice gives. If we take the nodes to be equally spaced and $L(f) = \int_a^b f$, the resulting rules are the classical **Newton–Cotes formulae**.

**Definition 3.3** (Newton–Cotes)

Fix $\nu \geq 1$ and let $x_i = a + i(b-a)/\nu$ for $i = 0, \dots, \nu$. The closed Newton–Cotes

rule of order $\nu$ is

$$\int_a^b f(x)\,\mathrm{d}x \approx \sum_{i=0}^{\nu} a_i f(x_i), \qquad a_i = \int_a^b \ell_i(x)\,\mathrm{d}x.$$

**Example 3.4** (The Trapezoidal Rule)

With $\nu = 1$ we have $\ell_0(x) = (b-x)/(b-a)$ and $\ell_1(x) = (x-a)/(b-a)$, each of which integrates to $(b-a)/2$. Hence

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{2}\big(f(a)+f(b)\big),$$

which is just the area of the trapezium under the chord joining the two endpoints.

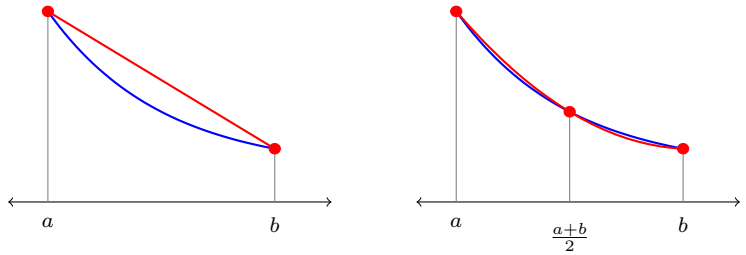
**Example 3.5** (Simpson's Rule)

With $\nu = 2$ and nodes $a$, $m = (a+b)/2$, $b$, a short computation gives $a_0 = a_2 = (b-a)/6$ and $a_1 = 2(b-a)/3$, so

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{b-a}{6}\left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right).$$

By construction this is exact for $f \in \mathcal{P}_2$. In fact it is exact for $f \in \mathcal{P}_3$: taking $[a,b] = [-1,1]$ for convenience, $\int_{-1}^{1} x^3\,\mathrm{d}x = 0$ and the rule returns $\frac{1}{3}(-1+0+1) = 0$ as well. This extra order 'for free' is a general feature of Newton–Cotes rules with an even number of subintervals, and comes from the symmetry of the nodes.

Geometrically, the trapezoidal rule replaces $f$ by a chord and Simpson's rule replaces it by a parabola through three points:




trapezoidal rule Simpson's rule

**Remark.** One might hope that taking $\nu$ larger and larger gives better and better rules. It does not: for $\nu \geq 8$ some of the weights $a_i$ become negative, and by $\nu = 30$ they are enormous, so that rounding errors are amplified catastrophically. This is the Runge phenomenon again, wearing a different hat. In practice one instead uses a *composite* rule: chop $[a,b]$ into many small subintervals and apply a low-order rule (usually the trapezoidal or Simpson's rule) on each.

## §3.2 Gaussian Quadrature

Now we use our remaining freedom. We work with a weighted integral

$$L(f) = \int_a^b w(x)f(x)\,\mathrm{d}x,$$

and seek **weights** $b_1, \dots, b_\nu$ and **nodes** $c_1, \dots, c_\nu \in [a, b]$ such that

$$\int_a^b w(x)f(x)\,\mathrm{d}x \approx \sum_{k=1}^{\nu} b_k f(c_k)$$

is exact for polynomials of as high a degree as possible. (The notation is traditional for this topic and clashes slightly with the rest of the course; we shall live with it.)

First, a limit on our ambitions.

**Proposition 3.6**

No choice of $\nu$ nodes and weights gives a formula exact for all $f \in \mathcal{P}_{2\nu}$.

*Proof.* Consider $q(x) = \prod_{k=1}^{\nu}(x - c_k)$, so that $q^2 \in \mathcal{P}_{2\nu}$. Then $\sum_k b_k q^2(c_k) = 0$, whereas

$$\int_a^b w(x)q(x)^2\,\mathrm{d}x > 0,$$

since $wq^2 \geq 0$ is continuous and not identically zero. □

So $2\nu - 1$ is the best we could possibly hope for, and we shall achieve it. The clue is in the proof: we need the nodes to be the roots of a polynomial that is orthogonal to everything of low degree. Before that, we must check that such roots are usable at all – they had better be real, distinct, and inside $(a, b)$.

**Theorem 3.7** (Zeros of Orthogonal Polynomials)

For $\nu \geq 1$, the monic orthogonal polynomial $p_\nu$ associated with $\langle f, g\rangle = \int_a^b wfg\,\mathrm{d}x$ has $\nu$ distinct real zeros, all lying in $(a, b)$.

*Proof.* Since $p_0 \equiv 1$ and $\langle p_\nu, p_0\rangle = 0$ for $\nu \geq 1$, we have

$$\int_a^b w(x)p_\nu(x)\,\mathrm{d}x = 0,$$

and as $w \geq 0$ is not identically zero, $p_\nu$ must change sign somewhere in $(a, b)$.

Let $\varphi_1, \dots, \varphi_m$ be the points of $(a, b)$ at which $p_\nu$ changes sign; we have just shown $m \geq 1$, and certainly $m \leq \nu$ since these are among the zeros of $p_\nu$. Put

$$q(x) = \prod_{j=1}^{m}(x - \varphi_j) \in \mathcal{P}_m.$$

Then $q$ changes sign at exactly the same points as $p_\nu$, so $qp_\nu$ has constant sign on $(a, b)$ and is not identically zero; hence $\langle q, p_\nu\rangle = \int_a^b wqp_\nu\,\mathrm{d}x \neq 0$. But $p_\nu$ is

orthogonal to every polynomial of degree less than $\nu$, so we must have $m \geq \nu$, and therefore $m = \nu$.

Thus $p_\nu$ has $\nu$ distinct sign changes in $(a, b)$, which accounts for all $\nu$ of its zeros; they are real, distinct, and lie in $(a, b)$. □

Before choosing the nodes, let us record what the interpolatory choice of weights gives for *any* nodes.

**Theorem 3.8** (Ordinary Quadrature)

Let $c_1, \ldots, c_\nu \in [a, b]$ be distinct, with fundamental Lagrange polynomials $\ell_1, \ldots, \ell_\nu$. If we choose

$$b_k = \int_a^b w(x)\ell_k(x)\,\mathrm{d}x,$$

then the quadrature formula is exact for all $f \in \mathcal{P}_{\nu-1}$.

*Proof.* For $f \in \mathcal{P}_{\nu-1}$, interpolation at $\nu$ points reproduces $f$ exactly, so $f = \sum_k f(c_k)\ell_k$. Integrating against $w$ gives $\int_a^b wf = \sum_k f(c_k)b_k$. □

And now the theorem that makes it all worthwhile.

**Theorem 3.9** (Gaussian Quadrature)

Choose the nodes $c_1, \ldots, c_\nu$ to be the zeros of the orthogonal polynomial $p_\nu$, and the weights $b_k$ as in Theorem 3.8. Then the quadrature formula

$$\int_a^b w(x)f(x)\,\mathrm{d}x \approx \sum_{k=1}^{\nu} b_k f(c_k)$$

is exact for all $f \in \mathcal{P}_{2\nu-1}$. Moreover every weight $b_k$ is strictly positive.

*Proof.* Let $f \in \mathcal{P}_{2\nu-1}$. Dividing by $p_\nu$ we may write

$$f = qp_\nu + r, \qquad q, r \in \mathcal{P}_{\nu-1}.$$

Since $p_\nu$ is orthogonal to all of $\mathcal{P}_{\nu-1}$, we have $\int_a^b wqp_\nu\,\mathrm{d}x = \langle q, p_\nu\rangle = 0$, so

$$\int_a^b w(x)f(x)\,\mathrm{d}x = \int_a^b w(x)r(x)\,\mathrm{d}x.$$

On the other side, each $c_k$ is a zero of $p_\nu$, so $f(c_k) = r(c_k)$ and hence

$$\sum_{k=1}^{\nu} b_k f(c_k) = \sum_{k=1}^{\nu} b_k r(c_k).$$

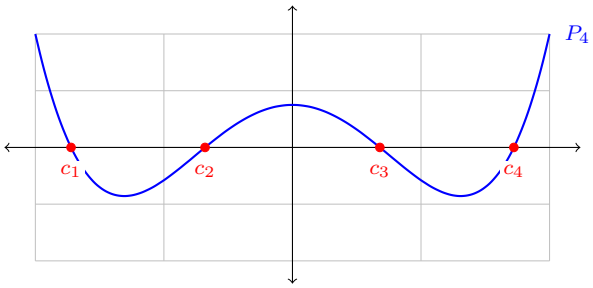
But $r \in \mathcal{P}_{\nu-1}$, so by Theorem 3.8 the two right-hand sides agree. Hence the formula is exact for $f$.

For positivity, apply exactness to $f = \ell_k^2 \in \mathcal{P}_{2\nu-2}$. Then

$$0 < \int_a^b w(x)\ell_k(x)^2 \,\mathrm{d}x = \sum_{j=1}^{\nu} b_j \ell_k(c_j)^2 = \sum_{j=1}^{\nu} b_j \delta_{jk} = b_k. \qquad \square$$

Together with Proposition 3.6, this says that Gaussian quadrature is optimal: $2\nu - 1$ is achieved and $2\nu$ is impossible. The positivity of the weights is not a footnote either – it is what makes the rule numerically stable, since it means there is no cancellation between the terms and rounding errors cannot be amplified.

For the Legendre weight $w \equiv 1$ on $[-1, 1]$ the nodes are the zeros of the Legendre polynomials. Here is $P_4$ with its four zeros marked; these are the nodes of the four-point Gauss–Legendre rule, which is exact for all polynomials of degree up to 7.




**Example 3.10** (Gauss–Legendre Rules)

For $w \equiv 1$ on $[-1, 1]$ the first few rules are as follows.

| $\nu$ | nodes $c_k$ | weights $b_k$ | exact on |
|---|---|---|---|
| 1 | $0$ | $2$ | $\mathcal{P}_1$ |
| 2 | $\pm 1/\sqrt{3}$ | $1,\ 1$ | $\mathcal{P}_3$ |
| 3 | $0,\ \pm\sqrt{3/5}$ | $8/9,\ 5/9,\ 5/9$ | $\mathcal{P}_5$ |
| 4 | $\pm 0.3399810,\ \pm 0.8611363$ | $0.6521452,\ 0.3478548$ | $\mathcal{P}_7$ |

Let us check the case $\nu = 2$ directly. The nodes are the zeros of $p_2 = x^2 - \frac{1}{3}$, namely $\pm 1/\sqrt{3}$. The rule $\int_{-1}^{1} f \approx f(-1/\sqrt{3}) + f(1/\sqrt{3})$ certainly integrates 1 and $x$ correctly. For $x^2$ it gives $2/3$, which is right; for $x^3$ it gives 0, which is right. And for $x^4$ it gives $2/9$ against the true value $2/5$ – so exactness stops at degree $3 = 2\nu - 1$, exactly as predicted.

**Example 3.11** (Gauss Beats Simpson)

Let us approximate $\int_0^1 e^x \,\mathrm{d}x = e - 1 = 1.718282$. Substituting $x = (t+1)/2$ turns this into $\frac{1}{2}\int_{-1}^{1} e^{(t+1)/2} \,\mathrm{d}t$, and the two-point Gauss rule gives

$$\tfrac{1}{2}\left(e^{(1-1/\sqrt{3})/2} + e^{(1+1/\sqrt{3})/2}\right) = \tfrac{1}{2}(1.235345 + 2.200455) = 1.717900,$$

an error of $3.8 \times 10^{-4}$, using *two* evaluations of $f$. Simpson's rule, using *three* evaluations, gives

$$\tfrac{1}{6}\left(1 + 4e^{1/2} + e\right) = 1.718861,$$

an error of $5.8\times10^{-4}$. The trapezoidal rule gives 1.859141, an error of 0.14. Gaussian quadrature does more with less.

**Remark.** The price of Gaussian quadrature is that the nodes are irrational and must be looked up or computed (they are eigenvalues of the tridiagonal matrix formed from the three-term recurrence coefficients, which is how one actually computes them). Also, unlike a composite Newton–Cotes rule, refining from $\nu$ to $\nu+1$ points reuses none of the old function evaluations. Both objections are real, and both have been engineered around; but the exactness result of Theorem 3.9 is why Gaussian quadrature remains the default for smooth integrands.

## §4 Expressing Errors in Terms of Derivatives

We now have a supply of approximations to linear functionals, each of them exact on some space $\mathcal{P}_k$ of polynomials. What we do not yet have is a systematic way to say how large the error is when $f$ is *not* a polynomial. The Peano kernel theorem provides exactly that: it converts the statement 'the error annihilates $\mathcal{P}_k$' into an exact integral formula for the error in terms of $f^{(k+1)}$.

The setup is as follows. We have a linear functional $L$, an approximation $\sum_{i=0}^{n} a_i L_i(f)$ built from simpler functionals $L_i$, and the error functional

$$e_L(f) = L(f) - \sum_{i=0}^{n} a_i L_i(f),$$

which is again linear. We suppose that $e_L(f) = 0$ for all $f \in \mathcal{P}_k$; we say that $e_L$ **annihilates** $\mathcal{P}_k$.

**Example 4.1**

Suppose we approximate $L(f) = f(\beta)$ by the (rather silly) formula

$$L(f) \approx f(\alpha) + \frac{\beta-\alpha}{2}\big(f'(\alpha)+f'(\beta)\big), \qquad \alpha \neq \beta.$$

Then

$$e_L(f) = f(\beta) - f(\alpha) - \frac{\beta-\alpha}{2}\big(f'(\alpha)+f'(\beta)\big).$$

One checks directly that this vanishes for $f = 1, x, x^2$: for $f = x^2$, the left-hand piece is $\beta^2-\alpha^2$ and the right-hand piece is $(\beta-\alpha)(\alpha+\beta)$, which agree. It does not vanish for $f = x^3$. So $e_L$ annihilates $\mathcal{P}_2$.

We want a bound of the form

$$|e_L(f)| \le c_L \left\| f^{(k+1)} \right\|_\infty,$$

with $c_L$ as small as possible. For this we need $[a,b]$ to be finite, so that the suprema exist.

**Definition 4.2** (Sharp Constant)

The constant $c_L$ is **sharp** if for every $\varepsilon > 0$ there is some $f_\varepsilon \in C^{k+1}[a,b]$ with

$$|e_L(f_\varepsilon)| \geq (c_L - \varepsilon) \left\| f_\varepsilon^{(k+1)} \right\|_\infty .$$

## §4.1 The Peano Kernel Theorem

The tool is Taylor's theorem with integral remainder,

$$f(x) = f(a) + (x-a)f'(a) + \cdots + \frac{(x-a)^k}{k!} f^{(k)}(a) + \frac{1}{k!}\int_a^x (x-\theta)^k f^{(k+1)}(\theta)\,\mathrm{d}\theta.$$

The awkwardness is that $x$ appears in the limit of the integral, which stops us from interchanging $e_L$ with $\int$. We fix this with a piece of notation.

**Definition 4.3** (Truncated Power Function)

For $k \geq 0$ we define

$$(x-\theta)_+^k = \begin{cases} (x-\theta)^k & x \geq \theta, \\ 0 & x < \theta. \end{cases}$$

Note that for $k \geq 1$ this is a $C^{k-1}$ function of $x$: it is smooth except at $x = \theta$, where the first $k-1$ derivatives all vanish from both sides. With it, the Taylor remainder becomes

$$\frac{1}{k!}\int_a^b (x-\theta)_+^k f^{(k+1)}(\theta)\,\mathrm{d}\theta,$$

where now the limits do not involve $x$.

**Theorem 4.4** (Peano Kernel Theorem)

Let $[a,b]$ be finite and let $e_L$ be a linear functional annihilating $\mathcal{P}_k$ which commutes with the integral in the sense below. Then for all $f \in C^{k+1}[a,b]$,

$$e_L(f) = \frac{1}{k!}\int_a^b K(\theta) f^{(k+1)}(\theta)\,\mathrm{d}\theta, \qquad K(\theta) = e_L\big((x-\theta)_+^k\big),$$

where $e_L$ acts on $(x-\theta)_+^k$ as a function of $x$, with $\theta$ held fixed.

*Proof.* Write $f = T + R$ where $T \in \mathcal{P}_k$ is the Taylor polynomial of $f$ about $a$ of degree $k$ and

$$R(x) = \frac{1}{k!}\int_a^b (x-\theta)_+^k f^{(k+1)}(\theta)\,\mathrm{d}\theta$$

is the remainder. Since $e_L$ annihilates $\mathcal{P}_k$ we have $e_L(T) = 0$, so by linearity $e_L(f) = e_L(R)$. Interchanging $e_L$ with the integral,

$$e_L(f) = \frac{1}{k!}\int_a^b e_L\big((x-\theta)_+^k\big) f^{(k+1)}(\theta)\,\mathrm{d}\theta = \frac{1}{k!}\int_a^b K(\theta) f^{(k+1)}(\theta)\,\mathrm{d}\theta. \qquad \square$$

**Remark.** The interchange of $e_L$ with the integral is the only step needing care. For the functionals we use in this course – point values, point values of derivatives, and integrals, together with finite linear combinations of these – the interchange is valid and can be

verified directly in each case (for integrals it is Fubini, for point evaluations it is trivial, and for derivatives one differentiates under the integral sign). A sufficiently determined pure mathematician can construct linear functionals for which it fails, but we shall not meet any.

The point of the theorem is that $K$ depends only on $L$ and not at all on $f$. So having computed $K$ once, we get a whole family of bounds by applying Hölder's inequality in different ways:

$$|e_L(f)| \leq \frac{1}{k!} \begin{cases} \left(\displaystyle\int_a^b |K(\theta)| \, \mathrm{d}\theta\right) \left\|f^{(k+1)}\right\|_\infty, \\ \left(\displaystyle\int_a^b |K(\theta)|^2 \, \mathrm{d}\theta\right)^{1/2} \left\|f^{(k+1)}\right\|_2, \\ \|K\|_\infty \left\|f^{(k+1)}\right\|_1. \end{cases}$$

These constants can be shown to be sharp. And do not forget the factor $1/k!$ when computing $c_L$ – it is the most commonly dropped piece of the whole formula.

There is one very common situation in which we can say considerably more.

**Proposition 4.5** (Sign-Definite Kernels)

If $K$ does not change sign on $[a,b]$, then for every $f \in C^{k+1}[a,b]$ there is some $\xi \in (a,b)$, depending on $f$, with

$$e_L(f) = \frac{1}{k!}\left(\int_a^b K(\theta)\,\mathrm{d}\theta\right) f^{(k+1)}(\xi).$$

Moreover the constant $c_L = \frac{1}{k!}\left|\int_a^b K\right|$ in the $\|\cdot\|_\infty$ bound is attained, by $f(x) = x^{k+1}$.

*Proof.* Since $K$ has constant sign, the integral mean value theorem applies to $\int_a^b K f^{(k+1)}$ and yields $f^{(k+1)}(\xi)\int_a^b K$ for some $\xi$. For the second statement, $f = x^{k+1}$ has $f^{(k+1)} \equiv (k+1)!$ constant, so the inequality $\left|\int K f^{(k+1)}\right| \leq \int |K| \cdot \left\|f^{(k+1)}\right\|_\infty$ is an equality. □

## §4.2 Worked Kernels

Peano kernels are computed by brute force: substitute $(x-\theta)_+^k$ into the formula for $e_L$, and remember that the truncated power vanishes when its argument is negative, so the answer is piecewise.

**Example 4.6** (The Trapezoidal Rule)

Take $[a,b] = [0,1]$ and

$$e_L(f) = \int_0^1 f(x)\,\mathrm{d}x - \tfrac{1}{2}\big(f(0)+f(1)\big),$$

which annihilates $\mathcal{P}_1$, so $k = 1$. Then for $\theta \in [0, 1]$,

$$K(\theta) = \int_0^1 (x-\theta)_+ \, \mathrm{d}x - \tfrac{1}{2}\big((0-\theta)_+ + (1-\theta)_+\big) = \int_\theta^1 (x-\theta)\, \mathrm{d}x - \tfrac{1}{2}(1-\theta),$$

since $(0-\theta)_+ = 0$ for $\theta \geq 0$. Hence

$$K(\theta) = \frac{(1-\theta)^2}{2} - \frac{1-\theta}{2} = -\frac{\theta(1-\theta)}{2}.$$

This is a downward parabola vanishing at both endpoints, so $K \leq 0$ throughout and Proposition 4.5 applies. We have $\int_0^1 K = -\frac{1}{2}(\frac{1}{2} - \frac{1}{3}) = -\frac{1}{12}$, so

$$e_L(f) = -\tfrac{1}{12} f''(\xi), \qquad |e_L(f)| \leq \tfrac{1}{12} \left\| f'' \right\|_\infty,$$

which is the classical trapezoidal rule error, now derived rather than conjured.

**Example 4.7** (Simpson's Rule)

Take $[a, b] = [-1, 1]$ and

$$e_L(f) = \int_{-1}^1 f(x)\, \mathrm{d}x - \tfrac{1}{3}\big(f(-1) + 4f(0) + f(1)\big).$$

We saw that this annihilates $\mathcal{P}_3$, so we may take $k = 3$. For $\theta \in [0, 1]$ only the $f(1)$ term survives the truncation, and

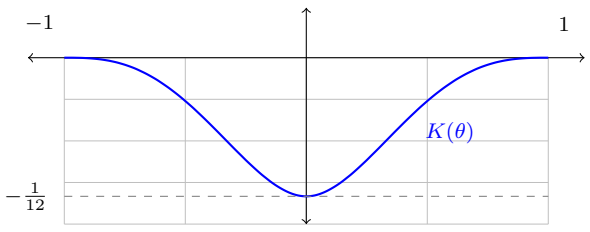
$$K(\theta) = \int_\theta^1 (x-\theta)^3 \, \mathrm{d}x - \tfrac{1}{3}(1-\theta)^3 = \frac{(1-\theta)^4}{4} - \frac{(1-\theta)^3}{3}.$$

For $\theta \in [-1, 0]$ the $f(0)$ term also contributes, giving

$$K(\theta) = \frac{(1-\theta)^4}{4} - \frac{1}{3}\left(4(-\theta)^3 + (1-\theta)^3\right).$$

One checks that the two branches agree at $\theta = 0$, both giving $\frac{1}{4} - \frac{1}{3} = -\frac{1}{12}$, and that $K(\pm 1) = 0$. In fact $K$ is symmetric, $K(-\theta) = K(\theta)$, and negative on $(-1, 1)$.

Here is the kernel of Example 4.7, plotted on $[-1, 1]$.




Since $K \leq 0$ we may use Proposition 4.5. Computing the integral,

$$\int_{-1}^1 K(\theta)\, \mathrm{d}\theta = 2\int_0^1 \left(\frac{(1-\theta)^4}{4} - \frac{(1-\theta)^3}{3}\right) \mathrm{d}\theta = 2\left(\frac{1}{20} - \frac{1}{12}\right) = -\frac{1}{15},$$

and hence, with $k = 3$ so that $1/k! = 1/6$,

$$e_L(f) = -\frac{1}{90} f^{(4)}(\xi), \qquad |e_L(f)| \leq \frac{1}{90} \left\| f^{(4)} \right\|_\infty$$

for some $\xi \in (-1,1)$. Rescaling to a general interval gives the familiar $-\frac{(b-a)^5}{2880} f^{(4)}(\xi)$.

The theorem applies just as well to functionals that have nothing to do with integration.

**Example 4.8** (A Derivative Formula)

Suppose we approximate $f'(0)$ by the one-sided formula

$$f'(0) \approx -\tfrac{3}{2}f(0) + 2f(1) - \tfrac{1}{2}f(2),$$

so that $e_L(f) = f'(0) + \frac{3}{2}f(0) - 2f(1) + \frac{1}{2}f(2)$ on $[0,2]$. Checking on $1, x, x^2$: for $f = 1$ we get $0 + \frac{3}{2} - 2 + \frac{1}{2} = 0$; for $f = x$ we get $1 + 0 - 2 + 1 = 0$; for $f = x^2$ we get $0 + 0 - 2 + 2 = 0$. For $f = x^3$ we get $0 + 0 - 2 + 4 = 2 \neq 0$. So $e_L$ annihilates $\mathcal{P}_2$ and $k = 2$.

The kernel is $K(\theta) = e_L\big((x-\theta)_+^2\big)$. Since $\frac{d}{dx}(x-\theta)_+^2 = 2(x-\theta)_+$, the derivative term contributes $2(0-\theta)_+ = 0$ for $\theta \geq 0$, as does the $f(0)$ term. So

$$K(\theta) = -2(1-\theta)_+^2 + \tfrac{1}{2}(2-\theta)_+^2 = \begin{cases} 2\theta - \frac{3}{2}\theta^2, & \theta \in [0,1], \\ \frac{1}{2}(2-\theta)^2, & \theta \in [1,2]. \end{cases}$$

(The two branches agree at $\theta = 1$, both giving $\frac{1}{2}$.) Both branches are non-negative, so $K \geq 0$ and Proposition 4.5 applies. We have

$$\int_0^2 K(\theta)\,\mathrm{d}\theta = \int_0^1 \left(2\theta - \tfrac{3}{2}\theta^2\right)\,\mathrm{d}\theta + \int_1^2 \tfrac{1}{2}(2-\theta)^2\,\mathrm{d}\theta = \tfrac{1}{2} + \tfrac{1}{6} = \tfrac{2}{3},$$

and hence, remembering the factor $1/k! = 1/2$,

$$e_L(f) = \tfrac{1}{3}f'''(\xi), \qquad |e_L(f)| \leq \tfrac{1}{3}\left\|f'''\right\|_\infty$$

for some $\xi \in (0,2)$.

**Remark.** A final subtlety. If $e_L$ annihilates $\mathcal{P}_k$ then it also annihilates $\mathcal{P}_j$ for every $j < k$, so we get a *different* kernel $K_j(\theta) = e_L((x-\theta)_+^j)$ and a different error formula

$$e_L(f) = \frac{1}{j!}\int_a^b K_j(\theta) f^{(j+1)}(\theta)\,\mathrm{d}\theta$$

valid for $f \in C^{j+1}[a,b]$. Usually taking $j = k$ is best, as it extracts the most information. But if $f$ happens to be only $C^{j+1}$ and not $C^{k+1}$ – and integrands with limited smoothness are common in practice – then the lower-order kernel is the one that applies. Simpson's rule with a merely twice-differentiable integrand is a case in point.

## §5 Ordinary Differential Equations

We now turn to what is probably the most important application of numerical analysis. We wish to approximate the solution $y : [0,\infty) \to \mathbb{R}^N$ of the initial value problem

$$y' = f(t,y), \qquad y(0) = y_0,$$

where $f : \mathbb{R} \times \mathbb{R}^N \to \mathbb{R}^N$. Note that we allow $y$ to be vector-valued, which costs us nothing notationally and buys us all higher-order scalar equations, since $y'' = g(t,y,y')$ becomes a first-order system in $(y,y')$.

We shall always assume that $f$ is nice enough for the problem to make sense. In particular, we assume $f$ is Lipschitz in its second argument, which by the Picard–Lindelöf theorem guarantees that a unique solution exists; and where we Taylor expand, we assume as much smoothness as the expansion requires. This is standard practice and is not usually where the interesting difficulties lie.

The idea is to replace the continuum of values $y(t)$ by a discrete sequence $y_0, y_1, y_2, \dots$ with $y_n \approx y(t_n)$, where $0 = t_0 < t_1 < \cdots$. We shall almost always take a constant step size, $t_n = nh$. The two questions we must ask of any such scheme are: does $y_n$ actually approach $y(t_n)$ as $h \to 0$, and how fast?

## §5.1 One-Step Methods

**Definition 5.1** (One-Step Method)

A **one-step method** is a recurrence of the form

$$y_{n+1} = \varphi_h(t_n, y_n),$$

in which $y_{n+1}$ depends only on $t_n$, $y_n$, $h$ and $f$.

The simplest of all is obtained by truncating the Taylor series of $y$ after one term.

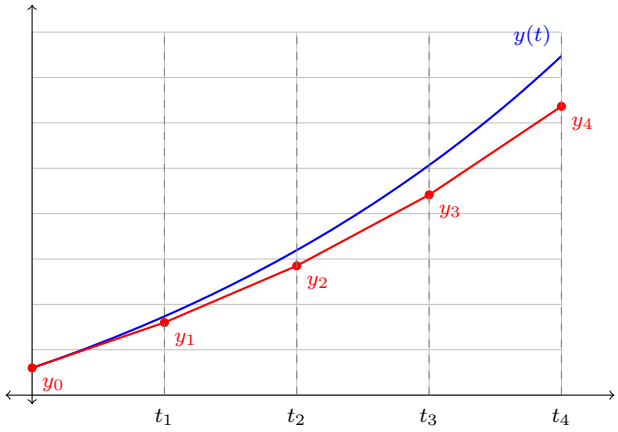
**Definition 5.2** (Euler's Method)

**Euler's method** is the one-step method

$$y_{n+1} = y_n + hf(t_n, y_n).$$

The idea is transparent: from the point $(t_n, y_n)$, the differential equation tells us the slope of the solution curve through that point, so we follow that straight line for a time $h$ and see where we end up. The resulting approximation is a polygon, and each segment of it is tangent to *some* solution curve of the equation – just not, in general, the one we want.

Here is Euler's method applied to $y' = y$, $y(0) = 1$, with $h = 1/4$. The true solution $y(t) = e^t$ is in blue and the Euler polygon in red.




Two things are visible in the picture. The Euler solution lags behind the true one, and the gap grows. What we need to know is whether the gap shrinks to nothing as we

shrink $h$.

**Definition 5.3** (Convergence)

A method is **convergent** if, for every $t^* > 0$ and every initial value problem with sufficiently nice $f$,

$$\lim_{h\to 0} \max_{n=0,1,\dots,\lfloor t^*/h \rfloor} \|y_n - y(t_n)\| = 0,$$

where $y_n$ is the numerical solution computed with step size $h$.

Note what this is asking: not that a fixed number of steps become accurate, but that the error at a fixed *time* $t^*$, reached in $\lfloor t^*/h \rfloor$ steps, goes to zero. Since the number of steps grows like $1/h$, the errors made at each step must not be allowed to accumulate too badly. This is the whole content of the following theorem.

**Theorem 5.4** (Convergence of Euler's Method)

Suppose $f$ is Lipschitz in its second argument on $[0, t^*] \times \mathbb{R}^N$, that is, there is $\lambda \geq 0$ with

$$\|f(t, v) - f(t, w)\| \leq \lambda \|v - w\|$$

for all $t \in [0, t^*]$ and $v, w \in \mathbb{R}^N$. Then Euler's method converges, and indeed the error is $\mathcal{O}(h)$ uniformly on $[0, t^*]$.

*Proof.* The Lipschitz condition guarantees by Picard–Lindelöf that a unique solution $y$ exists on $[0, t^*]$, so the statement makes sense. Write $e_n = y_n - y(t_n)$ for the (global) error at step $n$, and note $e_0 = 0$.

Taylor's theorem gives $y(t_{n+1}) = y(t_n) + hy'(t_n) + \mathcal{O}(h^2) = y(t_n) + hf(t_n, y(t_n)) + \mathcal{O}(h^2)$, where the $\mathcal{O}(h^2)$ is uniform in $n$ because $y''$ is continuous on the compact interval $[0, t^*]$. Subtracting this from the Euler step,

$$\begin{aligned}
\|e_{n+1}\| &= \big\|y_n + hf(t_n, y_n) - y(t_n) - hf(t_n, y(t_n)) - \mathcal{O}(h^2)\big\| \\
&\leq \|e_n\| + h\,\|f(t_n, y_n) - f(t_n, y(t_n))\| + ch^2 \\
&\leq (1 + h\lambda)\,\|e_n\| + ch^2
\end{aligned}$$

for some constant $c$ independent of $n$ and $h$. This is a linear recurrence, and unwinding it (using $e_0 = 0$) gives

$$\|e_{n+1}\| \leq ch^2 \sum_{j=0}^{n} (1 + h\lambda)^j = ch^2\, \frac{(1 + h\lambda)^{n+1} - 1}{h\lambda} \leq \frac{ch}{\lambda}(1 + h\lambda)^{n+1}.$$

Now we use $1 + x \leq e^x$, valid for all real $x$, to get $(1 + h\lambda)^{n+1} \leq e^{(n+1)h\lambda} = e^{\lambda t_{n+1}} \leq e^{\lambda t^*}$. Hence

$$\|e_{n+1}\| \leq \frac{c\, e^{\lambda t^*}}{\lambda} h$$

for every $n$ with $t_{n+1} \leq t^*$. The bound is independent of $n$ and tends to 0 with $h$, which is convergence with $\mathcal{O}(h)$ error. □

**Remark.** The factor $e^{\lambda t^*}$ is the price of accumulating $\mathcal{O}(1/h)$ errors of size $\mathcal{O}(h^2)$ each, with each error subsequently amplified by the flow. It is exponentially bad in $t^*$, which

is a genuine feature of the problem rather than an artefact of the proof: two nearby solutions of $y' = \lambda y$ really do separate exponentially. When $\lambda \to 0$ the bound looks singular, but $\frac{1}{\lambda}(e^{\lambda t^*} - 1) \to t^*$, so all is well.

The rate $\mathcal{O}(h)$ is what we mean by saying Euler's method has *order* 1. The general definition is local: we ask how much error a single step introduces, assuming that we started that step from the exact solution.

**Definition 5.5** (Order)

A one-step method $y_{n+1} = \varphi_h(t_n, y_n)$ has **order** $p$ if

$$y(t_{n+1}) - \varphi_h(t_n, y(t_n)) = \mathcal{O}(h^{p+1})$$

as $h \to 0$, for all sufficiently smooth $f$.

The exponent $p+1$ rather than $p$ is deliberate: as in the proof above, a *local* error of $\mathcal{O}(h^{p+1})$ made at each of $\mathcal{O}(1/h)$ steps accumulates to a *global* error of $\mathcal{O}(h^p)$. So a method of order $p$ has global error $\mathcal{O}(h^p)$, and higher order means we can take larger steps for a given accuracy.

**Example 5.6**

For Euler's method, $y(t_{n+1}) - y(t_n) - hf(t_n, y(t_n)) = \frac{1}{2}h^2 y''(t_n) + \mathcal{O}(h^3) = \mathcal{O}(h^2)$, so $p = 1$.

## §5.2 The Theta Method

We got Euler's method by evaluating $f$ at the left end of the interval $[t_n, t_{n+1}]$. We could equally evaluate at the right end, or at any weighted average of the two.

**Definition 5.7** (Theta Method)

For $\theta \in [0, 1]$, the **theta method** is

$$y_{n+1} = y_n + h\Big(\theta f(t_n, y_n) + (1-\theta) f(t_{n+1}, y_{n+1})\Big).$$

Taking $\theta = 1$ recovers Euler's method. Taking $\theta = 0$ gives the **backward Euler method**

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}),$$

and $\theta = \frac{1}{2}$ gives the **trapezoidal rule**

$$y_{n+1} = y_n + \tfrac{h}{2}\big(f(t_n, y_n) + f(t_{n+1}, y_{n+1})\big).$$

For $\theta \neq 1$ the unknown $y_{n+1}$ appears on both sides, so the method is **implicit**: each step requires solving a (generally nonlinear) equation. That is a real cost, and one might wonder why we would pay it. There are two reasons. The first is order.

**Proposition 5.8** (Order of the Theta Method)

The theta method has order 1, except when $\theta = \frac{1}{2}$, in which case it has order 2.

*Proof.* Assume $y_n = y(t_n)$ and expand everything about $t_n$, writing $y = y(t_n)$, $y' = y'(t_n)$ and so on. We have

$$y(t_{n+1}) = y + hy' + \tfrac{1}{2}h^2y'' + \tfrac{1}{6}h^3y''' + \mathcal{O}(h^4),$$

while $f(t_n, y(t_n)) = y'$ and, since $y'(t_{n+1}) = y' + hy'' + \tfrac{1}{2}h^2y''' + \mathcal{O}(h^3)$,

$$h\Big(\theta y' + (1-\theta)y'(t_{n+1})\Big) = hy' + (1-\theta)h^2y'' + \tfrac{1}{2}(1-\theta)h^3y''' + \mathcal{O}(h^4).$$

Subtracting, the local error is

$$\left(\theta - \tfrac{1}{2}\right)h^2y'' + \left(\tfrac{\theta}{2} - \tfrac{1}{3}\right)h^3y''' + \mathcal{O}(h^4).$$

The $h^2$ term vanishes precisely when $\theta = \tfrac{1}{2}$; and when it does, the coefficient of $h^3$ is $\tfrac{1}{4} - \tfrac{1}{3} = -\tfrac{1}{12} \neq 0$. So the order is 1 in general and 2 for $\theta = \tfrac{1}{2}$. □

The second reason for tolerating implicitness is *stability*, which will occupy us in Section 6. For the moment we simply record that implicit methods can be far better behaved on hard problems.

## §5.3 Multistep Methods

Euler's method throws away everything it has computed except the most recent value. Since we have $y_0, \dots, y_n$ sitting in memory, it is natural to use several of them.

**Definition 5.9** (Multistep Method)

An ***s*-step method** is a recurrence

$$\sum_{\ell=0}^{s} \rho_\ell\, y_{n+\ell} = h\sum_{\ell=0}^{s} \sigma_\ell\, f(t_{n+\ell}, y_{n+\ell}), \qquad n = 0, 1, \dots,$$

normalised so that $\rho_s = 1$. The method is **explicit** if $\sigma_s = 0$ and **implicit** otherwise.

To start such a method we need $s$ initial values $y_0, \dots, y_{s-1}$, of which the problem supplies only one; the rest are generated by some one-step method (typically a Runge–Kutta method of matching order). It is convenient to package the coefficients into two polynomials

$$\rho(w) = \sum_{\ell=0}^{s} \rho_\ell w^\ell, \qquad \sigma(w) = \sum_{\ell=0}^{s} \sigma_\ell w^\ell,$$

which will turn out to encode both the accuracy and the stability of the method.

**Example 5.10**

Euler's method is the one-step method with $\rho(w) = w - 1$, $\sigma(w) = 1$. Backward Euler has $\rho(w) = w - 1$, $\sigma(w) = w$. The trapezoidal rule has $\rho(w) = w - 1$, $\sigma(w) = \tfrac{1}{2}(w + 1)$. A genuine two-step example is the **Adams–Bashforth** method

$$y_{n+2} - y_{n+1} = h\left(\tfrac{3}{2}f(t_{n+1}, y_{n+1}) - \tfrac{1}{2}f(t_n, y_n)\right),$$

with $\rho(w) = w^2 - w$ and $\sigma(w) = \frac{3}{2}w - \frac{1}{2}$; and an implicit one is the two-step **Adams–Moulton** method

$$y_{n+2} - y_{n+1} = h\left(\tfrac{5}{12}f(t_{n+2}, y_{n+2}) + \tfrac{2}{3}f(t_{n+1}, y_{n+1}) - \tfrac{1}{12}f(t_n, y_n)\right).$$

**Definition 5.11** (Order of a Multistep Method)

The multistep method above has **order** $p$ if

$$\sum_{\ell=0}^{s} \rho_\ell\, y(t_{n+\ell}) - h\sum_{\ell=0}^{s} \sigma_\ell\, y'(t_{n+\ell}) = \mathcal{O}(h^{p+1})$$

as $h \to 0$, for all sufficiently smooth functions $y$.

Checking this by Taylor expansion each time would be tedious. Happily, the conditions can be packaged into a single elegant statement about $\rho$ and $\sigma$.

**Theorem 5.12** (Order Conditions)

The multistep method has order $p$ if and only if

$$\rho(e^z) - z\sigma(e^z) = \mathcal{O}(z^{p+1}) \qquad \text{as } z \to 0.$$

*Proof.* Expand everything about $t_n$. Writing $y^{(k)} = y^{(k)}(t_n)$ and $t_{n+\ell} = t_n + \ell h$,

$$y(t_{n+\ell}) = \sum_{k=0}^{\infty} \frac{(\ell h)^k}{k!} y^{(k)}, \qquad y'(t_{n+\ell}) = \sum_{k=1}^{\infty} \frac{(\ell h)^{k-1}}{(k-1)!} y^{(k)}.$$

Substituting into the definition and collecting powers of $h$,

$$\sum_\ell \rho_\ell y(t_{n+\ell}) - h\sum_\ell \sigma_\ell y'(t_{n+\ell}) = \left(\sum_\ell \rho_\ell\right) y + \sum_{k=1}^{\infty} \frac{1}{k!}\left(\sum_\ell \ell^k \rho_\ell - k\sum_\ell \ell^{k-1}\sigma_\ell\right) h^k y^{(k)}.$$

Since $y$ is arbitrary, this is $\mathcal{O}(h^{p+1})$ precisely when

$$\sum_\ell \rho_\ell = 0 \quad \text{and} \quad \sum_\ell \ell^k \rho_\ell = k\sum_\ell \ell^{k-1}\sigma_\ell \quad \text{for } k = 1, \dots, p. \qquad (*)$$

On the other hand, substituting $e^{\ell z} = \sum_k (\ell z)^k / k!$,

$$\begin{aligned} \rho(e^z) - z\sigma(e^z) &= \sum_\ell \rho_\ell e^{\ell z} - z\sum_\ell \sigma_\ell e^{\ell z} \\ &= \left(\sum_\ell \rho_\ell\right) + \sum_{k=1}^{\infty} \frac{1}{k!}\left(\sum_\ell \ell^k \rho_\ell - k\sum_\ell \ell^{k-1}\sigma_\ell\right) z^k, \end{aligned}$$

which is $\mathcal{O}(z^{p+1})$ under exactly the same conditions $(*)$. □

**Example 5.13**

For the two-step Adams–Bashforth method, $\rho(w) = w^2 - w$ and $\sigma(w) = \frac{3}{2}w - \frac{1}{2}$. Then

$$\rho(e^z) = e^{2z} - e^z = z + \tfrac{3}{2}z^2 + \tfrac{7}{6}z^3 + \mathcal{O}(z^4),$$
$$z\sigma(e^z) = z\left(\tfrac{3}{2}e^z - \tfrac{1}{2}\right) = z + \tfrac{3}{2}z^2 + \tfrac{3}{4}z^3 + \mathcal{O}(z^4),$$

so $\rho(e^z) - z\sigma(e^z) = \frac{5}{12}z^3 + \mathcal{O}(z^4)$. Hence the method has order 2, and the coefficient $\frac{5}{12}$ is its *error constant*.

## §5.4 The Root Condition and Dahlquist's Theorem

High order is not enough. Here is a cautionary example.

**Example 5.14**

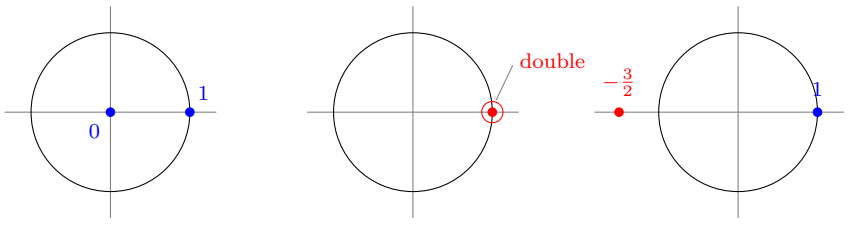
Consider the two-step method $y_{n+2} - 2y_{n+1} + y_n = 0$, that is $\rho(w) = (w-1)^2$ and $\sigma \equiv 0$. Then $\rho(e^z) - z\sigma(e^z) = (e^z - 1)^2 = z^2 + \mathcal{O}(z^3)$, so the method has order 1. But it does not involve $f$ at all – the numerical solution is the arithmetic progression $y_n = y_0 + n(y_1 - y_0)$ regardless of the differential equation. It certainly does not converge.

What is wrong is that the recurrence itself, ignoring the right-hand side, has solutions that grow. The homogeneous recurrence $\sum_\ell \rho_\ell y_{n+\ell} = 0$ has solutions $y_n = w^n$ where $\rho(w) = 0$, and repeated roots produce solutions $nw^n$, $n^2w^n$ and so on. For these to stay bounded we need the following.

**Definition 5.15** (Root Condition)

A polynomial $\rho$ obeys the **root condition** if all of its zeros lie in the closed unit disc $|w| \le 1$, and every zero on the unit circle $|w| = 1$ is simple.

Note that $\rho(1) = \sum_\ell \rho_\ell = 0$ is forced by order $p \ge 1$, so $w = 1$ is always a zero; the root condition says that it must be a simple one and that no other zero may escape the disc.




$\rho(w) = w^2 - w$ satisfied | $\rho(w) = (w-1)^2$ violated | $\rho(w) = (w-1)(w+\frac{3}{2})$ violated

The remarkable fact is that order and the root condition are not merely necessary for convergence – together they are sufficient.

**Theorem 5.16** (Dahlquist Equivalence Theorem)

A multistep method is convergent if and only if it has order $p \ge 1$ and its polynomial $\rho$ obeys the root condition.

We shall not prove this; the proof is long and was not given in lectures. But it is worth

appreciating what it does for us. Convergence is a statement about all initial value problems and all step sizes – an infinite amount of checking. The theorem replaces it by two finite algebraic checks on two polynomials, both of which can be done by hand in a minute.

**Example 5.17**

The two-step Adams–Bashforth method has $\rho(w) = w^2 - w = w(w-1)$, whose roots are 0 and 1; the root condition holds, and we computed the order to be 2. So it converges.

**Example 5.18**

The method $y_{n+2} - 3y_{n+1} + 2y_n = h\big(f(t_{n+1}, y_{n+1}) - 2f(t_n, y_n)\big)$ has $\rho(w) = w^2 - 3w + 2 = (w-1)(w-2)$. The root $w = 2$ lies outside the unit disc, so the method fails the root condition and does not converge, however good its order may be. (In fact its order is 2.)

## §5.5 Constructing Good Multistep Methods

We would like a recipe for producing convergent methods of high order. Theorem 5.12 tells us to make $\rho(e^z) - z\sigma(e^z)$ vanish to high order at $z = 0$; equivalently, substituting $w = e^z$ so that $z = \log w$, we want

$$\rho(w) - \sigma(w)\log w = \mathcal{O}\big(|w-1|^{p+1}\big) \qquad \text{as } w \to 1.$$

So: *choose $\rho$ first*, subject to $\rho(1) = 0$ and the root condition, and then choose $\sigma$ to be the truncation of the Taylor expansion of $\rho(w)/\log w$ about $w = 1$. If $s = \deg \rho$, we truncate to degree $s-1$ for an explicit method and degree $s$ for an implicit one.

**Method 5.19** (Adams Methods)

Take $\rho(w) = w^{s-1}(w-1)$, the simplest choice satisfying the root condition (roots 0 with multiplicity $s-1$, and a simple root at 1). The corresponding methods have the form

$$y_{n+s} - y_{n+s-1} = h\sum_{\ell=0}^{s} \sigma_\ell f(t_{n+\ell}, y_{n+\ell}).$$

The explicit choice ($\sigma_s = 0$) gives the **Adams–Bashforth** methods, of order $s$; the implicit choice gives the **Adams–Moulton** methods, of order $s+1$.

To see the orders claimed, note that $\rho(w)/\log w$ is analytic near $w = 1$ (the zero of $\log w$ at $w = 1$ is cancelled by the factor $w - 1$). If $\sigma$ is its Taylor polynomial of degree $m$ about $w = 1$, then $\rho(w)/\log w - \sigma(w) = \mathcal{O}(|w-1|^{m+1})$ and hence

$$\rho(w) - \sigma(w)\log w = \mathcal{O}\big(|w-1|^{m+1}\,|\log w|\big) = \mathcal{O}\big(|w-1|^{m+2}\big),$$

giving order $m+1$. With $m = s-1$ we get order $s$, and with $m = s$ order $s+1$.

**Example 5.20** (Small Adams Methods)

For $s = 1$: $\rho(w) = w - 1$, and Adams–Bashforth is Euler's method (order 1), while Adams–Moulton is the trapezoidal rule (order 2).

For $s = 2$: $\rho(w) = w^2 - w$. The expansion of $\rho(w)/\log w$ about $w = 1$ begins $1 + \frac{3}{2}(w-1) + \frac{5}{12}(w-1)^2 + \cdots$. Truncating to degree 1 gives $\sigma(w) = 1 + \frac{3}{2}(w-1) = \frac{3}{2}w - \frac{1}{2}$, which is the two-step Adams–Bashforth method of order 2; keeping the quadratic term gives the two-step Adams–Moulton method of order 3, which written out is

$$y_{n+2} - y_{n+1} = h\left(\tfrac{5}{12}f_{n+2} + \tfrac{2}{3}f_{n+1} - \tfrac{1}{12}f_n\right),$$

where we abbreviate $f_m = f(t_m, y_m)$.

There is a second family worth knowing, obtained by fixing $\sigma$ instead of $\rho$.

**Definition 5.21** (Backward Differentiation Formulae)

A **BDF method** is an $s$-step method of the form

$$\sum_{\ell=0}^{s} \rho_\ell y_{n+\ell} = h\sigma_s f(t_{n+s}, y_{n+s}),$$

so that $f$ is evaluated at one point only, at the new time level.

**Theorem 5.22** (BDF Coefficients)

The $s$-step BDF method of order $s$ has

$$\rho(w) = \sigma_s \sum_{\ell=1}^{s} \frac{1}{\ell} w^{s-\ell}(w-1)^\ell, \qquad \sigma_s = \left(\sum_{\ell=1}^{s} \frac{1}{\ell}\right)^{-1}.$$

*Proof.* We want $\rho(w) - \sigma_s w^s \log w = \mathcal{O}(|w-1|^{s+1})$. Now

$$\log w = -\log\left(\frac{1}{w}\right) = -\log\left(1 - \frac{w-1}{w}\right) = \sum_{\ell=1}^{\infty} \frac{1}{\ell}\left(\frac{w-1}{w}\right)^\ell,$$

so that

$$\sigma_s w^s \log w = \sigma_s \sum_{\ell=1}^{\infty} \frac{1}{\ell} w^{s-\ell}(w-1)^\ell.$$

Truncating this series after $\ell = s$ commits an error $\mathcal{O}(|w-1|^{s+1})$, and the truncation is exactly the stated $\rho$, which is a polynomial of degree $s$. Finally, $\rho_s = 1$ requires the coefficient of $w^s$ to be 1; each term contributes $\sigma_s/\ell$, giving $\sigma_s \sum_{\ell=1}^{s} 1/\ell = 1$. □

**Example 5.23**

For $s = 1$ we get $\sigma_1 = 1$ and $\rho(w) = w - 1$: backward Euler. For $s = 2$, $\sigma_2 = (1 + \frac{1}{2})^{-1} = \frac{2}{3}$ and

$$\rho(w) = \tfrac{2}{3}\left(w(w-1) + \tfrac{1}{2}(w-1)^2\right) = w^2 - \tfrac{4}{3}w + \tfrac{1}{3},$$

giving the method

$$y_{n+2} - \tfrac{4}{3}y_{n+1} + \tfrac{1}{3}y_n = \tfrac{2}{3}hf(t_{n+2}, y_{n+2}).$$

Its $\rho$ has roots 1 and $\frac{1}{3}$, so the root condition holds and the method converges.

**Remark.** The BDF construction produces a $\rho$ that we did not get to choose, so we must check the root condition afterwards – and it can fail. In fact the $s$-step BDF method satisfies the root condition precisely when $s \leq 6$. Nobody uses BDF7. The reason to care about these methods at all is their exceptional stability on stiff problems, which we come to shortly.

## §5.6 Runge–Kutta Methods

Multistep methods get high order by remembering the past. Runge–Kutta methods get it a different way: they take several trial steps *within* the current interval, and combine them. The motivation comes from quadrature. If $f$ happened not to depend on $y$ at all, then

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t)\,\mathrm{d}t,$$

and we could apply any quadrature rule we liked to the integral – Gaussian quadrature, say – to get

$$y_{n+1} = y_n + h\sum_{\ell=1}^{\nu} b_\ell\, f(t_n + c_\ell h),$$

where the $b_\ell$ and $c_\ell$ come from the rule rescaled to $[0,1]$. When $f$ does depend on $y$ we would need $y(t_n + c_\ell h)$, which we do not have. The Runge–Kutta idea is to manufacture approximations to those intermediate values out of the data we do have.

**Definition 5.24** (Runge–Kutta Method)

A **$\nu$-stage Runge–Kutta method** is

$$k_\ell = f\left(t_n + c_\ell h,\; y_n + h\sum_{j=1}^{\nu} a_{\ell j} k_j\right), \quad \ell = 1,\dots,\nu, \qquad y_{n+1} = y_n + h\sum_{\ell=1}^{\nu} b_\ell k_\ell,$$

where $c_\ell = \sum_{j=1}^{\nu} a_{\ell j}$. The method is **explicit** if $a_{\ell j} = 0$ whenever $j \geq \ell$, so that each $k_\ell$ is computed from those before it; otherwise it is implicit.

The coefficients are conventionally displayed in a **Butcher tableau**

$$\begin{array}{c|c} c & A \\ \hline & b^\top \end{array} \quad = \quad \begin{array}{c|ccc} c_1 & a_{11} & \cdots & a_{1\nu} \\ c_2 & a_{21} & \cdots & a_{2\nu} \\ \vdots & \vdots & & \vdots \\ c_\nu & a_{\nu 1} & \cdots & a_{\nu\nu} \\ \hline & b_1 & \cdots & b_\nu \end{array}$$

in which the row sums of $A$ are the entries of $c$. An explicit method has $A$ strictly lower triangular, so its tableau has a triangle of zeros (usually left blank) on and above the diagonal.

**Example 5.25** (Euler and the Trapezoidal Rule)

Euler's method is the one-stage explicit method with tableau

$$\begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}$$

and the trapezoidal rule is the two-stage implicit method

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1 & \frac{1}{2} & \frac{1}{2} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}.$$

Let us see how the order conditions arise, in the smallest interesting case.

**Example 5.26** (Two-Stage Explicit Methods)

Take $\nu = 2$ with $a_{21} = c_2$, so that

$$k_1 = f(t_n, y_n), \qquad k_2 = f(t_n + c_2 h,\ y_n + c_2 h k_1),$$

and $y_{n+1} = y_n + h(b_1 k_1 + b_2 k_2)$. Assume $y_n = y(t_n)$ and expand. Taylor's theorem in two variables gives

$$k_2 = f + c_2 h\,(f_t + f_y f) + \mathcal{O}(h^2),$$

where $f$ and its partials are evaluated at $(t_n, y_n)$. On the other hand, differentiating $y' = f(t, y)$ gives $y'' = f_t + f_y f$, so

$$y_n + h(b_1 k_1 + b_2 k_2) = y + h(b_1 + b_2)y' + h^2 b_2 c_2 y'' + \mathcal{O}(h^3),$$

whereas $y(t_{n+1}) = y + hy' + \frac{1}{2}h^2 y'' + \mathcal{O}(h^3)$. Matching coefficients, the local error is

$$h(1 - b_1 - b_2)y' + \tfrac{h^2}{2}(1 - 2b_2 c_2)y'' + \mathcal{O}(h^3),$$

so we obtain order 2 exactly when

$$b_1 + b_2 = 1, \qquad b_2 c_2 = \tfrac{1}{2}.$$

Two popular solutions are $b = (\frac{1}{2}, \frac{1}{2})$, $c_2 = 1$ (the *improved Euler* or Heun method) and $b = (0, 1)$, $c_2 = \frac{1}{2}$ (the *midpoint* method).

**Proposition 5.27**

No two-stage explicit Runge–Kutta method has order 3.

*Proof.* It suffices to exhibit one problem for which order 3 fails. Take $y' = y$, $y(0) = 1$, with exact solution $y(t) = e^t$. Then $k_1 = e^{t_n}$ and $k_2 = e^{t_n}(1 + c_2 h)$, so the local error is

$$e^{t_{n+1}} - e^{t_n}\left(1 + h(b_1 + b_2) + h^2 b_2 c_2\right) = e^{t_n}\left(e^h - 1 - h(b_1 + b_2) - h^2 b_2 c_2\right).$$

Expanding $e^h$, the $h^3$ coefficient is $\frac{1}{6}e^{t_n}$, which no choice of $b_\ell, c_\ell$ can remove since they do not appear in it. So the local error is $\Theta(h^3)$ at best. □

The most celebrated Runge–Kutta method of all is the four-stage, fourth-order one, which people simply call 'RK4'.

**Example 5.28** (The Classical RK4)

The method

$$\begin{aligned} k_1 &= f(t_n, y_n), \\ k_2 &= f(t_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_1), \\ k_3 &= f(t_n + \tfrac{h}{2},\ y_n + \tfrac{h}{2}k_2), \\ k_4 &= f(t_n + h,\ y_n + hk_3), \\ y_{n+1} &= y_n + \tfrac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \end{aligned}$$

has order 4. Its Butcher tableau is

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}.$$

Notice that the weights $\frac{1}{6}, \frac{1}{3}, \frac{1}{3}, \frac{1}{6}$ and the nodes $0, \frac{1}{2}, \frac{1}{2}, 1$ are exactly Simpson's rule on $[0,1]$ (with the midpoint counted twice). This is no accident: if $f$ is independent of $y$ then $k_2 = k_3$ and the method *is* Simpson's rule.

**Remark.** Verifying that RK4 has order 4 by direct Taylor expansion is a genuinely unpleasant computation – one must match multivariate derivatives of $f$ up to third order – and we shall not do it here. The systematic treatment is due to Butcher and is organised by rooted trees. What is worth knowing is the general shape of the answer: an explicit $\nu$-stage method has order at most $\nu$, with equality possible only for $\nu \leq 4$. To get order 5 one needs 6 stages, which is one reason RK4 remains so popular.

**Remark.** Implicit Runge–Kutta methods can do much better. The $\nu$-stage *Gauss–Legendre* method, whose nodes $c_\ell$ are the Gaussian quadrature nodes on $[0,1]$, has order $2\nu$ – the maximum possible – which is precisely the exactness result of Theorem 3.9 reappearing in a new guise. The two-stage version, with

$$c = \left(\tfrac{1}{2} - \tfrac{\sqrt{3}}{6},\ \tfrac{1}{2} + \tfrac{\sqrt{3}}{6}\right), \qquad b = \left(\tfrac{1}{2}, \tfrac{1}{2}\right),$$

has order 4 using only two evaluations of $f$ per step, at the cost of solving an implicit system.

## §6 Stiff Equations and Stability

Everything so far has been about behaviour as $h \to 0$. In practice we do not take $h \to 0$; we take $h$ as large as we dare, and the question is how large that is. For some problems the answer is dictated not by accuracy but by *stability*, and the discrepancy can be enormous.

**Example 6.1**

Consider the scalar problem $y' = -100y$, $y(0) = 1$, whose solution $e^{-100t}$ has decayed to $10^{-44}$ by $t = 1$. Euler's method gives $y_n = (1-100h)^n$. If $h = 0.1$ then $y_n = (-9)^n$, which oscillates with wildly growing amplitude: at $t = 1$ we get $y_{10} = 3.5 \times 10^9$ instead of essentially zero. We need $|1 - 100h| < 1$, that is $h < 0.02$, purely to avoid nonsense – even though the solution itself is utterly featureless after $t = 0.05$ and any reasonable notion of accuracy would permit much larger steps.

**Definition 6.2** (Stiffness)

An initial value problem is **stiff** if the step size of a numerical method must be kept much smaller than accuracy alone would demand, in order to keep the numerical solution bounded.

This is a deliberately informal definition, and stiffness is a property of the problem-and-method pair rather than of the problem alone. Stiffness typically arises in systems $y' = Ay$ whose eigenvalues have very different magnitudes: the fast-decaying components are numerically irrelevant after a moment, but they continue to dictate the step size forever. Such systems are the norm in chemical kinetics and in the method of lines for parabolic PDEs.

## §6.1 Linear Stability Domains

To analyse this we use a test problem simple enough to compute with and rich enough to be informative.

**Definition 6.3** (Linear Stability Domain)

Apply a numerical method with constant step $h$ to the scalar test equation

$$y' = \lambda y, \qquad y(0) = 1, \qquad \lambda \in \mathbb{C}.$$

The **linear stability domain** of the method is

$$\mathcal{D} = \{h\lambda \in \mathbb{C} : y_n \to 0 \text{ as } n \to \infty\}.$$

The exact solution $e^{\lambda t}$ tends to 0 precisely when $\operatorname{Re}\lambda < 0$, so we would like $\mathcal{D}$ to contain the whole left half-plane.

**Definition 6.4** (A-stability)

A method is **A-stable** if $\mathbb{C}^- = \{z \in \mathbb{C} : \operatorname{Re} z < 0\} \subseteq \mathcal{D}$.

Let us compute $\mathcal{D}$ for the theta method. Applying it to $y' = \lambda y$ gives

$$y_{n+1} = y_n + h\lambda\big(\theta y_n + (1-\theta)y_{n+1}\big) \implies y_{n+1} = \frac{1+\theta z}{1-(1-\theta)z} y_n, \quad z = h\lambda.$$

So $y_n = r(z)^n$ with $r(z) = \frac{1+\theta z}{1-(1-\theta)z}$, and $y_n \to 0$ if and only if $|r(z)| < 1$.

**Example 6.5** (Euler's Method)

Here $\theta = 1$ and $r(z) = 1 + z$, so

$$\mathcal{D} = \{z : |1 + z| < 1\},$$

the open disc of radius 1 centred at $-1$. This is a small set: it fails to contain, for instance, $z = -3$, which is exactly the instability we saw above. Euler's method is not A-stable.

**Example 6.6** (Backward Euler)

Here $\theta = 0$ and $r(z) = (1 - z)^{-1}$, so

$$\mathcal{D} = \{z : |1 - z| > 1\},$$

the *exterior* of the open disc of radius 1 centred at $+1$. This contains the whole left half-plane, so backward Euler is A-stable. In fact $\mathcal{D}$ is much bigger than $\mathbb{C}^-$, which means backward Euler damps some components that the true solution does not damp – a form of inaccuracy, but a benign one.

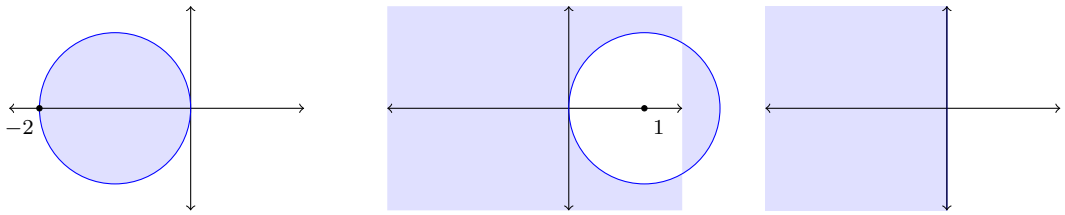
**Example 6.7** (The Trapezoidal Rule)

Here $\theta = \frac{1}{2}$ and

$$r(z) = \frac{1 + z/2}{1 - z/2} = \frac{2 + z}{2 - z},$$

so $|r(z)| < 1$ if and only if $|2 + z| < |2 - z|$, that is, if and only if $z$ is closer to $-2$ than to 2. Hence $\mathcal{D} = \mathbb{C}^-$ exactly: the trapezoidal rule is A-stable, and its stability domain is precisely right – it damps exactly what should be damped.

Here are the three domains, with the stable region shaded.




Euler
$|1 + z| < 1$

backward Euler
$|1 - z| > 1$

trapezoidal rule
$\operatorname{Re} z < 0$

The contrast is striking. The explicit method has a bounded stability domain, so for any fixed $\lambda$ with $\operatorname{Re}\lambda \ll 0$ there is a hard ceiling on $h$; the two implicit methods have no such ceiling at all. This is the pay-off that justifies the extra work of solving an implicit equation at each step, and it is why stiff problems are solved with implicit methods.

**Remark.** The trapezoidal rule is A-stable but its $r(z)$ satisfies $|r(z)| \to 1$ as $|z| \to \infty$, so very stiff components are barely damped – they merely oscillate in sign. Backward Euler has $r(z) \to 0$, and kills them outright. A method with $r(\infty) = 0$ is called *L-stable*, and for genuinely stiff problems this is what one wants.

## §6.2 Stability of Multistep and Runge–Kutta Methods

For a multistep method the analysis is a little different, since the recurrence is of order $s$. Applying the method to $y' = \lambda y$ with $z = h\lambda$ gives

$$\sum_{\ell=0}^{s} \rho_\ell y_{n+\ell} = z \sum_{\ell=0}^{s} \sigma_\ell y_{n+\ell}, \qquad \text{i.e.} \qquad \sum_{\ell=0}^{s} (\rho_\ell - z\sigma_\ell) y_{n+\ell} = 0.$$

This is a linear recurrence with characteristic polynomial $\rho(w) - z\sigma(w)$, and its solutions tend to zero exactly when all the roots of that polynomial lie strictly inside the unit disc. So

$$\mathcal{D} = \{z \in \mathbb{C} : \text{all roots } w \text{ of } \rho(w) - z\sigma(w) \text{ satisfy } |w| < 1\}.$$

Notice that $z = 0$ gives back the root condition (in its non-strict form), which is reassuring: linear stability at $z = 0$ is exactly convergence.

This is harder to work with, and the news is bad.

**Theorem 6.8** (Second Dahlquist Barrier)

No A-stable multistep method has order greater than 2. Among the A-stable multistep methods of order 2, the trapezoidal rule has the smallest error constant.

We do not prove this. It is a genuine obstruction rather than a failure of ingenuity, and it forces a choice: either accept low order, or weaken the demand for A-stability, or leave the multistep world altogether.

The second option is what is done in practice. Many stiff systems have eigenvalues that are not merely in $\mathbb{C}^-$ but far from the imaginary axis, so it suffices to have a stability domain containing a wedge around the negative real axis. A method whose stability domain contains such a wedge $\{z : |\arg(-z)| < \alpha\}$ is called $A(\alpha)$-*stable*, and the BDF methods of order up to 6 all have this property with a decent $\alpha$. This is why they exist.

The third option is Runge–Kutta. Applying a Runge–Kutta method to $y' = \lambda y$ always gives $y_{n+1} = r(z)y_n$ for a rational function $r$, so $\mathcal{D} = \{z : |r(z)| < 1\}$; and there is no barrier theorem here.

**Example 6.9** (An A-stable Implicit Runge–Kutta Method)

Consider the two-stage implicit method with tableau

$$\begin{array}{c|cc} 0 & \frac{1}{4} & -\frac{1}{4} \\ \frac{2}{3} & \frac{1}{4} & \frac{5}{12} \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array},$$

that is,

$$k_1 = f\left(t_n, y_n + \tfrac{h}{4}(k_1 - k_2)\right), \quad k_2 = f\left(t_n + \tfrac{2h}{3}, y_n + \tfrac{h}{12}(3k_1 + 5k_2)\right),$$

with $y_{n+1} = y_n + \frac{h}{4}(k_1 + 3k_2)$. Applying this to $y' = \lambda y$ and eliminating $k_1, k_2$ gives $y_{n+1} = r(z)y_n$ with

$$r(z) = \frac{6 + 2z}{6 - 4z + z^2}.$$

We claim $\mathcal{D} \supseteq \mathbb{C}^-$. The denominator $z^2 - 4z + 6$ has roots $2 \pm i\sqrt{2}$, both in the right half-plane, so $r$ is analytic on $\overline{\mathbb{C}^-}$; and $r(z) \to 0$ as $|z| \to \infty$. By the maximum modulus principle, $|r|$ attains its maximum over the closed left half-plane on the boundary, so it suffices to check $|r(ix)| \leq 1$ for $x \in \mathbb{R}$. Now

$$|6 + 2ix|^2 = 36 + 4x^2, \qquad \left|6 - x^2 - 4ix\right|^2 = (6 - x^2)^2 + 16x^2 = 36 + 4x^2 + x^4,$$

so $|r(ix)|^2 = \frac{36+4x^2}{36+4x^2+x^4} \leq 1$, with equality only at $x = 0$. Hence $|r(z)| < 1$ throughout $\mathbb{C}^-$, and the method is A-stable. It can be shown that it also has order 3 – beating the Dahlquist barrier comfortably.

**Remark.** The same argument shows the $\nu$-stage Gauss–Legendre methods are all A-stable; the two-stage one has

$$r(z) = \frac{12 + 6z + z^2}{12 - 6z + z^2},$$

for which $|r(ix)| = 1$ for all real $x$, so its stability domain is exactly $\mathbb{C}^-$. Order 4 and exact stability, in two stages – at the price of solving a coupled implicit system at every step.

## §6.3 Implementation

We close the discussion of differential equations with a few words on what one actually does, since the theory above leaves two practical questions unanswered: how big should $h$ be, and how do we solve the implicit equations?

### Choosing the step size

A user of an ODE solver does not want to specify $h$; they want to specify an error tolerance $\varepsilon$ and have the solver work out $h$ for itself, varying it as the solution demands. To do that we need a cheap estimate of the local error at each step.

**Method 6.10** (The Milne Device)

Run two multistep methods of the same order side by side, one explicit (the *predictor*) and one implicit (the *corrector*). If the two methods have error constants $c_P$ and $c_C$, then

$$y^P_{n+1} \approx y(t_{n+1}) - c_P h^{p+1} y^{(p+1)}(t_n), \qquad y^C_{n+1} \approx y(t_{n+1}) - c_C h^{p+1} y^{(p+1)}(t_n).$$

Subtracting eliminates the unknown derivative,

$$h^{p+1} y^{(p+1)}(t_n) \approx \frac{y^P_{n+1} - y^C_{n+1}}{c_C - c_P},$$

and substituting back gives the estimate

$$y^C_{n+1} - y(t_{n+1}) \approx \frac{c_C}{c_C - c_P} \left(y^P_{n+1} - y^C_{n+1}\right)$$

for the local error of the corrector, computable from quantities we have already.

**Example 6.11**

With the two-step Adams–Bashforth method as predictor ($c_P = \frac{5}{12}$) and the trapezoidal rule as corrector ($c_C = -\frac{1}{12}$), the estimate is

$$y^C_{n+1} - y(t_{n+1}) \approx -\tfrac{1}{6}\left(y^P_{n+1} - y^C_{n+1}\right).$$

The predictor costs one extra evaluation of $f$, which is cheap, and it doubles as an excellent starting guess for the implicit corrector equation.

Given the estimate, the logic is straightforward: if it exceeds the tolerance $\varepsilon$, reject the step and retry with smaller $h$; if it is comfortably below (say below $\varepsilon/10$), accept the step and increase $h$; otherwise accept and carry on unchanged.

For Runge–Kutta methods there is no single error constant to exploit, and the analogous device is *embedded* Runge–Kutta: one uses two methods, of orders $p$ and $p+1$, sharing as many stages as possible so that the extra accuracy is nearly free.

**Example 6.12** (An Embedded Pair)

With

$$k_1 = f(t_n, y_n), \quad k_2 = f\left(t_n + \tfrac{h}{2}, y_n + \tfrac{h}{2}k_1\right), \quad k_3 = f\left(t_n + h, y_n - hk_1 + 2hk_2\right),$$

the combination $y^{[1]}_{n+1} = y_n + hk_2$ has order 2, while $y^{[2]}_{n+1} = y_n + \frac{h}{6}(k_1 + 4k_2 + k_3)$ has order 3. The difference $y^{[1]}_{n+1} - y^{[2]}_{n+1}$ estimates the error in the lower-order formula, at the cost of three function evaluations in total.

### Solving the implicit equations

An implicit method requires us to solve, at each step, an equation of the form

$$y_{n+s} = h\sigma_s f(t_{n+s}, y_{n+s}) + v,$$

where $v$ collects everything already known. The simplest approach is **functional iteration**,

$$y^{[j+1]}_{n+s} = h\sigma_s f\left(t_{n+s}, y^{[j]}_{n+s}\right) + v,$$

started from the predictor. This is a contraction provided $h\,|\sigma_s|\,\lambda < 1$, where $\lambda$ is the Lipschitz constant of $f$ – but that is precisely the sort of restriction on $h$ we adopted an implicit method to escape. So for stiff problems functional iteration is useless, and one uses the **Newton–Raphson** method instead: setting $\psi(y) = y - h\sigma_s f(t_{n+s}, y) - v$, iterate

$$y^{[j+1]}_{n+s} = y^{[j]}_{n+s} - \left(D\psi\left(y^{[j]}_{n+s}\right)\right)^{-1} \psi\left(y^{[j]}_{n+s}\right),$$

which converges quadratically from a good starting guess. Evaluating and inverting the Jacobian $D\psi = I - h\sigma_s D_y f$ at every iteration is expensive, so in practice one freezes it,

$$y^{[j+1]}_{n+s} = y^{[j]}_{n+s} - \left(D\psi\left(y^{[0]}_{n+s}\right)\right)^{-1} \psi\left(y^{[j]}_{n+s}\right),$$

recomputing only when convergence deteriorates. Either way we are left with a linear system to solve at every step, usually with the same matrix many times over – which is exactly the situation the next section is designed for.

# §7 Numerical Linear Algebra

Solving $A\mathbf{x} = \mathbf{b}$ is the single most common task in scientific computing, and one that we have already run into twice in this course – in the normal equations for least-squares fitting, and in the Newton iteration for implicit ODE methods. In pure linear algebra the problem is considered solved: $\mathbf{x} = A^{-1}\mathbf{b}$, and Cramer's rule even gives a formula. But Cramer's rule for a $20 \times 20$ system requires more arithmetic than has been performed in the history of the world, so we shall have to do better.

The strategy throughout is *factorisation*: write $A$ as a product of matrices for which the system is easy, solve a sequence of easy systems, and – crucially – reuse the factorisation if the same $A$ appears again with a different $\mathbf{b}$.

## §7.1 Triangular Systems

The easy case is a triangular matrix.

**Definition 7.1**

A square matrix $L$ is **lower triangular** if $L_{ij} = 0$ for $j > i$, and **unit** lower triangular if in addition $L_{ii} = 1$ for all $i$. Similarly $U$ is **upper triangular** if $U_{ij} = 0$ for $j < i$.

If $L$ is lower triangular and nonsingular, the system $L\mathbf{y} = \mathbf{b}$ is solved by *forward substitution*: the first equation gives $y_1 = b_1/L_{11}$, and having found $y_1, \dots, y_{i-1}$ the $i$-th equation gives

$$y_i = \frac{1}{L_{ii}} \left( b_i - \sum_{j<i} L_{ij} y_j \right).$$

This costs $\mathcal{O}(n^2)$ operations, one multiplication for each nonzero entry of $L$. Upper triangular systems are solved the same way from the bottom up, by *back substitution*.

## §7.2 LU Factorisation

**Definition 7.2** (LU Factorisation)

An **LU factorisation** of a square matrix $A$ is a factorisation $A = LU$ in which $L$ is unit lower triangular and $U$ is upper triangular.

Given such a factorisation, everything becomes cheap. To solve $A\mathbf{x} = \mathbf{b}$ we solve $L\mathbf{y} = \mathbf{b}$ by forward substitution and then $U\mathbf{x} = \mathbf{y}$ by back substitution, at a cost of $\mathcal{O}(n^2)$ for each new right-hand side. The determinant is

$$\det A = \det L \det U = \prod_{i=1}^{n} U_{ii},$$

so $A$ is singular precisely when some $U_{ii} = 0$.

How do we compute it? Write $\mathbf{l}_1, \dots, \mathbf{l}_n$ for the columns of $L$ and $\mathbf{u}_1^\top, \dots, \mathbf{u}_n^\top$ for the rows of $U$. Then

$$A = LU = \sum_{k=1}^{n} \mathbf{l}_k \mathbf{u}_k^\top,$$

an expression of $A$ as a sum of $n$ rank-one matrices. The point is that $\mathbf{l}_k$ has zeros in its first $k-1$ entries and $\mathbf{u}_k$ has zeros in its first $k-1$ entries, so $\mathbf{l}_k\mathbf{u}_k^\top$ has zeros in its first $k-1$ rows *and* its first $k-1$ columns. Consequently the first row and column of $A$ come entirely from the $k=1$ term, and we can read them off.

**Method 7.3** (LU by Outer Products)

Set $A_0 = A$. For $k = 1, 2, \dots, n$:

(i) let $\mathbf{u}_k^\top$ be the $k$-th row of $A_{k-1}$;

(ii) let $\mathbf{l}_k$ be the $k$-th column of $A_{k-1}$, divided by $(A_{k-1})_{kk}$ so that $(\mathbf{l}_k)_k = 1$;

(iii) set $A_k = A_{k-1} - \mathbf{l}_k\mathbf{u}_k^\top$.

Then $A = \sum_k \mathbf{l}_k\mathbf{u}_k^\top = LU$.

At step $k$ the dominant cost is forming the outer product, which touches an $(n-k+1)\times(n-k+1)$ block, so the total cost is $\sum_k (n-k+1)^2 \sim \frac{1}{3}n^3$ operations. This is the same $\mathcal{O}(n^3)$ as Gaussian elimination – indeed, the algorithm *is* Gaussian elimination, with the multipliers recorded in $L$ instead of discarded. The advantage of keeping them is that a second right-hand side then costs only $\mathcal{O}(n^2)$ rather than another $\mathcal{O}(n^3)$.

**Example 7.4**

Let

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}.$$

Step 1: $\mathbf{u}_1^\top = (2,1,1)$ and $\mathbf{l}_1 = (1,2,4)^\top$, so

$$A_1 = A - \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix}(2,1,1) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{pmatrix}.$$

Step 2: $\mathbf{u}_2^\top = (0,1,1)$ and $\mathbf{l}_2 = (0,1,3)^\top$, giving

$$A_2 = A_1 - \begin{pmatrix} 0 \\ 1 \\ 3 \end{pmatrix}(0,1,1) = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 2 \end{pmatrix}.$$

Step 3: $\mathbf{u}_3^\top = (0,0,2)$, $\mathbf{l}_3 = (0,0,1)^\top$. Hence

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix},$$

and $\det A = 2 \cdot 1 \cdot 2 = 4$.

The algorithm breaks down if $(A_{k-1})_{kk} = 0$, since we cannot divide by it. Sometimes this is unavoidable: the matrix $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ has no LU factorisation at all, as one sees immediately by multiplying out $\left(\begin{smallmatrix} 1 & 0 \\ \ell & 1 \end{smallmatrix}\right)\left(\begin{smallmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{smallmatrix}\right)$ and comparing top-left entries. The precise condition is the following.

**Theorem 7.5** (Existence and Uniqueness of LU)

Let $A$ be $n \times n$ and let $A^{(k)}$ denote its leading $k \times k$ principal submatrix. If $A^{(k)}$ is nonsingular for every $k = 1, \dots, n$, then $A$ has an LU factorisation, and it is unique.

*Proof.* *Uniqueness* first, and note that it does not need the hypothesis beyond $A$ being nonsingular. Suppose $A = LU = L'U'$ with $L, L'$ unit lower triangular and $U, U'$ upper triangular and nonsingular. Then $(L')^{-1}L = U'U^{-1}$. The left side is unit lower triangular (inverses and products of unit lower triangular matrices are unit lower triangular) and the right side is upper triangular. A matrix that is both is diagonal, and being unit lower triangular it must be $I$. Hence $L = L'$ and $U = U'$.

*Existence*, by induction on $k$, constructing the factorisation of $A^{(k)}$. For $k = 1$, take $L = (1)$ and $U = (A_{11})$, which is legitimate since $A^{(1)} \neq 0$. Suppose $A^{(k)} = L_k U_k$ with $L_k$ unit lower triangular and $U_k$ upper triangular; since $\det A^{(k)} = \det U_k \neq 0$, both factors are nonsingular. Write

$$A^{(k+1)} = \begin{pmatrix} A^{(k)} & \mathbf{b} \\ \mathbf{c}^\top & d \end{pmatrix},$$

and look for a factorisation

$$A^{(k+1)} = \begin{pmatrix} L_k & \mathbf{0} \\ \mathbf{x}^\top & 1 \end{pmatrix} \begin{pmatrix} U_k & \mathbf{y} \\ \mathbf{0}^\top & u \end{pmatrix} = \begin{pmatrix} L_k U_k & L_k \mathbf{y} \\ \mathbf{x}^\top U_k & \mathbf{x}^\top \mathbf{y} + u \end{pmatrix}.$$

The top-left block is already correct. The remaining equations are $L_k \mathbf{y} = \mathbf{b}$, $U_k^\top \mathbf{x} = \mathbf{c}$ and $u = d - \mathbf{x}^\top \mathbf{y}$, and each has a unique solution because $L_k$ and $U_k$ are nonsingular. This completes the induction, and taking $k = n$ gives the result. □

**Remark.** The hypothesis is sufficient but not necessary: the zero matrix has an LU factorisation (many of them) while all its principal minors vanish. What the theorem really says is that if the leading minors are nonsingular, the naive algorithm never has to divide by zero – and the pivots it uses are exactly $U_{kk} = \det A^{(k)} / \det A^{(k-1)}$.

## §7.3 Pivoting

A zero pivot is fatal, and – worse – a *small* pivot is nearly fatal, since dividing by it amplifies whatever rounding error is present. The cure for both is to reorder the rows.

**Method 7.6** (LU with Partial Pivoting)

At step $k$, before doing anything else, choose $p \geq k$ maximising $|(A_{k-1})_{pk}|$ and let $P_k$ be the permutation matrix exchanging rows $k$ and $p$. Then take $\mathbf{u}_k^\top$ to be the $k$-th row of $P_k A_{k-1}$, take

$$\mathbf{l}_k = \frac{1}{(P_k A_{k-1})_{kk}} \left(k\text{-th column of } P_k A_{k-1}\right),$$

and set $A_k = P_k A_{k-1} - \mathbf{l}_k \mathbf{u}_k^\top$.

Choosing the largest available entry guarantees that every entry of $\mathbf{l}_k$ has modulus at

most 1, so the multipliers never magnify errors. Unwinding the algorithm, one finds

$$(P_{n-1}\cdots P_1)A = \tilde{\mathbf{l}}_1\mathbf{u}_1^\top + \cdots + \tilde{\mathbf{l}}_n\mathbf{u}_n^\top, \qquad \tilde{\mathbf{l}}_k = P_{n-1}\cdots P_{k+1}\mathbf{l}_k,$$

and since the permutations applied to $\mathbf{l}_k$ only touch entries from position $k+1$ onwards, the vectors $\tilde{\mathbf{l}}_k$ still have zeros in their first $k-1$ places. So they are the columns of a unit lower triangular matrix, and we have proved the following.

**Theorem 7.7** (LU with Pivoting)

For any square matrix $A$ there is a permutation matrix $P$ such that $PA = LU$ with $L$ unit lower triangular and $U$ upper triangular.

*Proof.* Run the algorithm above, taking $P = P_{n-1}\cdots P_1$. The only remaining worry is a step at which the entire $k$-th column of $A_{k-1}$ (from row $k$ down) is zero; in that case there is nothing to eliminate, and we may simply take $P_k = I$, $\mathbf{l}_k = \mathbf{e}_k$ and $\mathbf{u}_k^\top$ the $k$-th row of $A_{k-1}$, which preserves everything we need. □

To solve $A\mathbf{x} = \mathbf{b}$ we then solve $L\mathbf{y} = P\mathbf{b}$ and $U\mathbf{x} = \mathbf{y}$; permuting a vector is free.

## §7.4 Symmetric Matrices and Cholesky

When $A$ is symmetric it is wasteful to compute both $L$ and $U$, since we ought to be able to get one from the other. The right factorisation is the following.

**Definition 7.8**

An $LDL^\top$ **factorisation** of a symmetric matrix $A$ is a factorisation $A = LDL^\top$ with $L$ unit lower triangular and $D$ diagonal.

Setting $U = DL^\top$ recovers an LU factorisation, so by Theorem 7.5 the $LDL^\top$ factorisation is unique when it exists. It is computed by exactly the same outer-product scheme: writing $A = \sum_k D_{kk}\mathbf{l}_k\mathbf{l}_k^\top$, we set $A_0 = A$, take $D_{kk} = (A_{k-1})_{kk}$, take $\mathbf{l}_k$ to be the $k$-th column of $A_{k-1}$ scaled so that $(\mathbf{l}_k)_k = 1$, and put $A_k = A_{k-1} - D_{kk}\mathbf{l}_k\mathbf{l}_k^\top$. Since we only need the lower half of each $A_k$, the work is halved.

**Example 7.9**

Take $A = \begin{pmatrix} 2 & 4 \\ 4 & 11 \end{pmatrix}$. Then $D_{11} = 2$ and $\mathbf{l}_1 = (1,2)^\top$, so

$$A_1 = \begin{pmatrix} 2 & 4 \\ 4 & 11 \end{pmatrix} - 2\begin{pmatrix} 1 \\ 2 \end{pmatrix}(1,2) = \begin{pmatrix} 0 & 0 \\ 0 & 3 \end{pmatrix},$$

whence $D_{22} = 3$, $\mathbf{l}_2 = (0,1)^\top$ and

$$A = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

**Definition 7.10**

A symmetric matrix $A$ is **positive definite** if $\mathbf{x}^\top A\mathbf{x} > 0$ for every $\mathbf{x} \neq \mathbf{0}$.

**Theorem 7.11**

A real symmetric matrix $A$ is positive definite if and only if it has an $LDL^\top$ factorisation in which every diagonal entry of $D$ is positive.

*Proof.* ($\Leftarrow$) If $A = LDL^\top$ with $D_{kk} > 0$, then for $\mathbf{x} \neq \mathbf{0}$ put $\mathbf{z} = L^\top\mathbf{x}$, which is nonzero since $L^\top$ is nonsingular. Then $\mathbf{x}^\top A\mathbf{x} = \mathbf{z}^\top D\mathbf{z} = \sum_k D_{kk} z_k^2 > 0$.

($\Rightarrow$) Suppose $A$ is positive definite. Every leading principal submatrix $A^{(k)}$ is then positive definite (test with vectors supported on the first $k$ coordinates), hence nonsingular, so by Theorem 7.5 the factorisation exists. Its diagonal entries satisfy $D_{kk} = \det A^{(k)} / \det A^{(k-1)}$, and all these determinants are positive because a positive definite matrix has positive determinant (its eigenvalues are all positive). Hence $D_{kk} > 0$. □

**Definition 7.12** (Cholesky Factorisation)

If $A$ is symmetric positive definite, write $D^{1/2}$ for the diagonal matrix with entries $\sqrt{D_{kk}}$ and set $\tilde{L} = LD^{1/2}$. Then

$$A = \tilde{L}\tilde{L}^\top$$

with $\tilde{L}$ lower triangular with positive diagonal. This is the **Cholesky factorisation**.

Cholesky is the workhorse for symmetric positive definite systems. It needs no pivoting – the theorem guarantees the pivots are positive, and one can show they cannot grow – it uses half the storage and half the arithmetic of LU, and it fails, in a numerically detectable way, precisely when $A$ is not positive definite. That last property makes it the standard test for positive definiteness.

## §7.5 Banded and Sparse Matrices

Large matrices arising in practice are rarely dense. Discretising a differential equation, for instance, couples each unknown only to its neighbours, and the resulting matrix has almost all entries zero. It would be a shame to spend $\mathcal{O}(n^3)$ operations on such a matrix, and we need not.

**Theorem 7.13** (Inheritance of Zeros)

Let $A = LU$ be an LU factorisation. If $A_{ij} = 0$ for all $j \leq m$ (that is, row $i$ of $A$ begins with $m$ zeros, with $m < i$), then $L_{ij} = 0$ for $j \leq m$; and symmetrically, leading zeros in the columns of $A$ above the diagonal are inherited by $U$.

*Proof.* We induct on $j$. Suppose $A_{i1} = 0$. Since $A_{i1} = \sum_k L_{ik}U_{k1} = L_{i1}U_{11}$ and $U_{11} \neq 0$, we get $L_{i1} = 0$. Now suppose $L_{i1} = \cdots = L_{i,j-1} = 0$ and $A_{ij} = 0$ with

$j < i$. Then

$$0 = A_{ij} = \sum_{k \le j} L_{ik} U_{kj} = L_{ij} U_{jj},$$

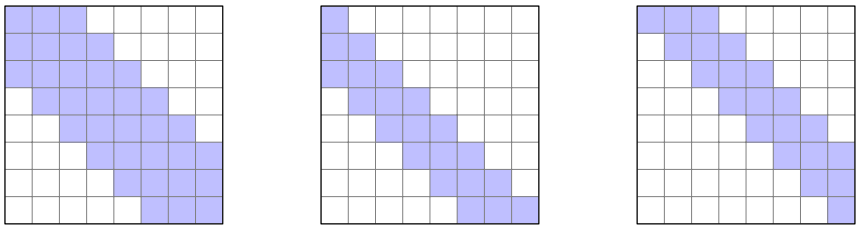
and $U_{jj} \neq 0$ gives $L_{ij} = 0$. The statement for $U$ follows by applying this to $A^\top$. □

**Definition 7.14** (Banded Matrix)

A matrix $A$ is **banded** with bandwidth $r$ if $A_{ij} = 0$ whenever $|i - j| > r$.

Combining the two: if $A$ is banded with bandwidth $r$, then row $i$ of $A$ begins with $i - r - 1$ zeros and so does row $i$ of $L$; likewise for $U$. So $L$ and $U$ are banded with the same bandwidth $r$, and the factorisation can be computed by touching only the band. The cost drops from $\mathcal{O}(n^3)$ to $\mathcal{O}(r^2 n)$, which for a tridiagonal matrix ($r = 1$) is $\mathcal{O}(n)$.

The picture is worth a glance: shaded cells are the ones that may be nonzero.



$A$ (bandwidth $r = 2$) $L$ $U$

**Remark.** For a general sparse matrix – one with many zeros but no particular pattern – the situation is more delicate, because elimination can create nonzeros where there were none. This is called *fill-in*, and minimising it by reordering rows and columns is a genuinely hard combinatorial problem, attacked in practice with graph-theoretic heuristics. Iterative methods, which never form a factorisation at all, are the other standard response. Both are beyond this course, and both are covered in Part II.

## §8 QR Factorisation and Least Squares

The LU factorisation is fast, but it is not always well behaved: the factors can be much larger than $A$ itself, and errors can grow accordingly. Orthogonal matrices, by contrast, preserve lengths exactly, and so cannot amplify anything. This makes them the tool of choice when accuracy matters more than speed – and, as we shall see, they are exactly what is needed for the least-squares problem.

**Definition 8.1**

A set of vectors $\{\mathbf{q}_i\}$ in $\mathbb{R}^m$ is **orthonormal** if $\mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$. A square matrix $Q$ is **orthogonal** if its columns are orthonormal, equivalently $Q^\top Q = I$, equivalently $Q^{-1} = Q^\top$.

The key property is that orthogonal matrices are isometries: $\|Q\mathbf{x}\|^2 = \mathbf{x}^\top Q^\top Q \mathbf{x} = \|\mathbf{x}\|^2$. In particular $|\det Q| = 1$, and multiplying by $Q$ can neither magnify nor shrink an error.

**Definition 8.2** (QR Factorisation)

Let $A$ be $m \times n$. A **QR factorisation** of $A$ is a factorisation $A = QR$ with $Q$ an $m \times m$ orthogonal matrix and $R$ an $m \times n$ upper triangular matrix. If $m \geq n$, a **reduced** (or skinny) QR factorisation is $A = QR$ with $Q$ an $m \times n$ matrix with orthonormal columns and $R$ an $n \times n$ upper triangular matrix.

If $A$ is square and nonsingular and $A = QR$, then $A\mathbf{x} = \mathbf{b}$ becomes $R\mathbf{x} = Q^{\top}\mathbf{b}$, a triangular system. So QR does everything LU does; it just costs about twice as much.

To see what a reduced QR factorisation *means*, write $\mathbf{a}_1, \dots, \mathbf{a}_n$ for the columns of $A$ and $\mathbf{q}_1, \dots, \mathbf{q}_n$ for those of $Q$. Comparing columns in $A = QR$ and using upper-triangularity,

$$\mathbf{a}_k = \sum_{j=1}^{k} R_{jk}\mathbf{q}_j.$$

So $\mathbf{q}_1, \dots, \mathbf{q}_k$ is an orthonormal basis for the span of $\mathbf{a}_1, \dots, \mathbf{a}_k$, for every $k$. A reduced QR factorisation is nothing other than an orthonormalisation of the columns of $A$, carried out one column at a time. That immediately suggests an algorithm.

## §8.1 Gram–Schmidt

**Method 8.3** (Gram–Schmidt)

Assume $m \geq n$ and that the columns of $A$ are linearly independent. Set

$$R_{11} = \|\mathbf{a}_1\|, \qquad \mathbf{q}_1 = \mathbf{a}_1/R_{11},$$

and having found $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$, set

$$R_{ij} = \mathbf{q}_i^{\top}\mathbf{a}_j \ (i < j), \qquad \mathbf{b}_j = \mathbf{a}_j - \sum_{i=1}^{j-1} R_{ij}\mathbf{q}_i, \qquad R_{jj} = \|\mathbf{b}_j\|, \qquad \mathbf{q}_j = \mathbf{b}_j/R_{jj}.$$

The vector $\mathbf{b}_j$ is $\mathbf{a}_j$ with its components along $\mathbf{q}_1, \dots, \mathbf{q}_{j-1}$ removed, so it is orthogonal to all of them; and it is nonzero because the columns of $A$ are independent. The cost is $\mathcal{O}(mn^2)$.

This is the algorithm one meets in a first linear algebra course, and it is perfectly correct. Numerically, however, it is disappointing: the computed $\mathbf{q}_j$ drift out of orthogonality, because each is built from all its predecessors and their small errors compound. (Rearranging the arithmetic gives the 'modified' Gram–Schmidt process, which is noticeably better, but still not ideal.) The trouble is philosophical as much as practical – we are building an orthogonal matrix by a procedure that is not itself orthogonal.

The alternative is *orthogonal triangularisation*: rather than orthogonalising the columns of $A$, multiply $A$ on the left by a sequence of orthogonal matrices, chosen to introduce zeros, until what is left is upper triangular. If

$$\Omega_k \cdots \Omega_1 A = R$$

with each $\Omega_i$ orthogonal, then $A = QR$ with $Q = (\Omega_k \cdots \Omega_1)^{\top}$, which is orthogonal by construction. We need a supply of orthogonal matrices that create zeros, and there are two standard choices.

## §8.2 Givens Rotations

**Definition 8.4** (Givens Rotation)

For $1 \leq p < q \leq m$ and $\theta \in (-\pi, \pi]$, the **Givens rotation** $\Omega^{[p,q]}(\theta)$ is the $m \times m$ matrix equal to the identity except that

$$\Omega_{pp} = \Omega_{qq} = \cos\theta, \qquad \Omega_{pq} = \sin\theta, \qquad \Omega_{qp} = -\sin\theta.$$

Geometrically this is a rotation of the plane spanned by $\mathbf{e}_p$ and $\mathbf{e}_q$, leaving everything else fixed; it is clearly orthogonal. For $m = 4$, say,

$$\Omega^{[1,2]} = \begin{pmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad \Omega^{[2,4]} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & 0 & \sin\theta \\ 0 & 0 & 1 & 0 \\ 0 & -\sin\theta & 0 & \cos\theta \end{pmatrix}.$$

**Proposition 8.5**

Let $A$ be $m \times n$, let $1 \leq p < q \leq m$ and $1 \leq j \leq n$. Then there is $\theta$ such that $(\Omega^{[p,q]}A)_{qj} = 0$. Moreover $\Omega^{[p,q]}A$ agrees with $A$ in every row except the $p$-th and $q$-th, and those two rows are linear combinations of the $p$-th and $q$-th rows of $A$.

*Proof.* The second and third statements hold for every $\theta$, directly from the definition. For the first, if $A_{pj} = A_{qj} = 0$ then any $\theta$ works. Otherwise take

$$\cos\theta = \frac{A_{pj}}{\sqrt{A_{pj}^2 + A_{qj}^2}}, \qquad \sin\theta = \frac{A_{qj}}{\sqrt{A_{pj}^2 + A_{qj}^2}},$$

which is a legitimate choice since the two squares sum to 1. Then

$$(\Omega^{[p,q]}A)_{qj} = -\sin\theta\, A_{pj} + \cos\theta\, A_{qj} = \frac{-A_{qj}A_{pj} + A_{pj}A_{qj}}{\sqrt{A_{pj}^2 + A_{qj}^2}} = 0. \qquad \square$$

To triangularise $A$ we apply rotations that zero the subdiagonal entries one at a time, in an order that does not destroy the zeros already made. Because $\Omega^{[p,q]}$ only alters rows $p$ and $q$, taking the pairs $[p,q]$ in lexicographic order works: we clear column 1 using $\Omega^{[1,2]}, \Omega^{[1,3]}, \ldots, \Omega^{[1,m]}$, then column 2 using $\Omega^{[2,3]}, \ldots$, and so on.

**Example 8.6**

For a $3 \times 3$ matrix, $\Omega^{[2,3]}\Omega^{[1,3]}\Omega^{[1,2]}A$ is upper triangular for suitable choices of the three angles.

The cost is again $\mathcal{O}(mn^2)$, so on a general matrix Givens has no advantage. Its virtue is *selectivity*: each rotation touches only two rows, so if $A$ already has many zeros we need only a few rotations. For an $n \times n$ upper Hessenberg matrix (zero below the first subdiagonal) just $n-1$ rotations suffice, giving a QR factorisation in $\mathcal{O}(n^2)$ operations. This is exactly the situation inside the QR algorithm for eigenvalues, which is why Givens rotations are not merely of historical interest.

## §8.3 Householder Reflections

For a dense matrix, we would rather clear a whole column at once. Reflections do this.

**Definition 8.7** (Householder Reflection)

For $\mathbf{u} \in \mathbb{R}^m \setminus \{\mathbf{0}\}$, the **Householder reflection** associated with $\mathbf{u}$ is

$$H = I - \frac{2\mathbf{u}\mathbf{u}^\top}{\mathbf{u}^\top \mathbf{u}}.$$

**Proposition 8.8**

$H$ is symmetric and orthogonal, with $H\mathbf{u} = -\mathbf{u}$ and $H\mathbf{v} = \mathbf{v}$ for every $\mathbf{v}$ orthogonal to $\mathbf{u}$. Moreover, if $\mathbf{a}, \mathbf{b} \in \mathbb{R}^m$ satisfy $\|\mathbf{a}\| = \|\mathbf{b}\|$ and $\mathbf{u} = \mathbf{a} - \mathbf{b}$ is nonzero, then $H\mathbf{a} = \mathbf{b}$.

*Proof.* Symmetry is clear. For orthogonality, writing $\alpha = \mathbf{u}^\top \mathbf{u}$,

$$H^\top H = H^2 = I - \frac{4\mathbf{u}\mathbf{u}^\top}{\alpha} + \frac{4\mathbf{u}(\mathbf{u}^\top \mathbf{u})\mathbf{u}^\top}{\alpha^2} = I.$$

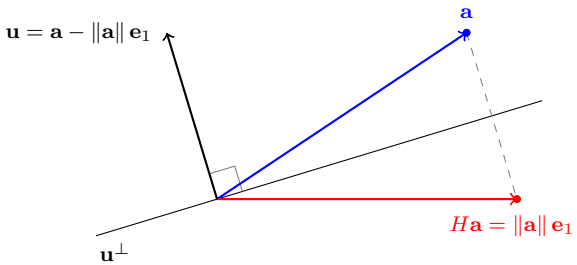
The formulas $H\mathbf{u} = -\mathbf{u}$ and $H\mathbf{v} = \mathbf{v}$ are immediate. For the last claim, put $\mathbf{u} = \mathbf{a} - \mathbf{b}$ and compute

$$\mathbf{u}^\top \mathbf{a} = \|\mathbf{a}\|^2 - \mathbf{b}^\top \mathbf{a}, \qquad \mathbf{u}^\top \mathbf{u} = \|\mathbf{a}\|^2 - 2\mathbf{b}^\top \mathbf{a} + \|\mathbf{b}\|^2 = 2\left(\|\mathbf{a}\|^2 - \mathbf{b}^\top \mathbf{a}\right),$$

using $\|\mathbf{a}\| = \|\mathbf{b}\|$. Hence $2\mathbf{u}^\top \mathbf{a}/\mathbf{u}^\top \mathbf{u} = 1$ and

$$H\mathbf{a} = \mathbf{a} - \mathbf{u} = \mathbf{b}. \qquad \square$$

So $H$ is the reflection of $\mathbb{R}^m$ in the hyperplane $\mathbf{u}^\perp$, and the last part says that any two vectors of equal length are reflections of each other in the hyperplane bisecting them. That is precisely what we need: taking $\mathbf{b} = \pm\|\mathbf{a}\|\,\mathbf{e}_1$ turns any vector into a multiple of $\mathbf{e}_1$.




**Method 8.9** (Householder QR)

At the first step, let $\mathbf{a}_1$ be the first column of $A$ and take

$$\mathbf{u} = \mathbf{a}_1 + \operatorname{sign}(A_{11})\,\|\mathbf{a}_1\|\,\mathbf{e}_1,$$

so that $H\mathbf{a}_1 = -\operatorname{sign}(A_{11})\,\|\mathbf{a}_1\|\,\mathbf{e}_1$ and $HA$ has zeros below the diagonal in its first column.

In general, suppose columns $1, \dots, k-1$ have been dealt with. Let $\tilde{\mathbf{a}}_k \in \mathbb{R}^{m-k+1}$ consist of the entries $A_{kk}, \dots, A_{mk}$ of the current $k$-th column, put

$$\tilde{\mathbf{u}} = \tilde{\mathbf{a}}_k + \operatorname{sign}(A_{kk}) \, \|\tilde{\mathbf{a}}_k\| \, \mathbf{e}_1 \in \mathbb{R}^{m-k+1},$$

and apply the block-diagonal orthogonal matrix

$$\begin{pmatrix} I_{k-1} & 0 \\ 0 & \tilde{H} \end{pmatrix}, \qquad \tilde{H} = I - \frac{2\tilde{\mathbf{u}}\tilde{\mathbf{u}}^\top}{\tilde{\mathbf{u}}^\top \tilde{\mathbf{u}}}.$$

The leading $I_{k-1}$ leaves the first $k-1$ rows – and hence the zeros already created – untouched.

The choice of sign deserves comment. Either sign gives a valid reflection, but if $A_{kk}$ and $\|\tilde{\mathbf{a}}_k\|$ nearly cancel then $\tilde{\mathbf{u}}$ is tiny and computed inaccurately. Choosing the sign of $A_{kk}$ makes the two terms reinforce rather than cancel, which is the numerically safe option.

In practice one never forms $\tilde{H}$ explicitly. Applying it to a vector $\mathbf{v}$ is just

$$\tilde{H}\mathbf{v} = \mathbf{v} - \frac{2(\tilde{\mathbf{u}}^\top \mathbf{v})}{\tilde{\mathbf{u}}^\top \tilde{\mathbf{u}}} \tilde{\mathbf{u}},$$

an inner product and an axpy, costing $\mathcal{O}(m)$ rather than $\mathcal{O}(m^2)$. Similarly $Q$ itself is normally stored as the list of vectors $\tilde{\mathbf{u}}$, and one computes $Q^\top \mathbf{b}$ by applying the reflections in turn. The total cost is $\mathcal{O}(mn^2)$.

**Example 8.10**

Suppose we have reached

$$A = \begin{pmatrix} 2 & 4 & 7 \\ 0 & 3 & -1 \\ 0 & 0 & 2 \\ 0 & 0 & 1 \\ 0 & 0 & -2 \end{pmatrix},$$

with the first two columns already processed, and we wish to clear the third. Here

$$\tilde{\mathbf{a}}_3 = \begin{pmatrix} 2 \\ 1 \\ -2 \end{pmatrix}, \qquad \|\tilde{\mathbf{a}}_3\| = 3, \qquad \tilde{\mathbf{u}} = \tilde{\mathbf{a}}_3 + 3\mathbf{e}_1 = \begin{pmatrix} 5 \\ 1 \\ -2 \end{pmatrix},$$

and $\tilde{\mathbf{u}}^\top \tilde{\mathbf{u}} = 30$, $\tilde{\mathbf{u}}^\top \tilde{\mathbf{a}}_3 = 10 + 1 + 4 = 15$. Hence

$$\tilde{H}\tilde{\mathbf{a}}_3 = \tilde{\mathbf{a}}_3 - \frac{2 \cdot 15}{30} \tilde{\mathbf{u}} = \begin{pmatrix} 2 \\ 1 \\ -2 \end{pmatrix} - \begin{pmatrix} 5 \\ 1 \\ -2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \\ 0 \end{pmatrix},$$

and the result is

$$R = \begin{pmatrix} 2 & 4 & 7 \\ 0 & 3 & -1 \\ 0 & 0 & -3 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

**Remark.** So which should one use? For a dense matrix, Householder: it clears an entire column with one reflection and is beautifully stable. For a matrix that is already nearly triangular, Givens: each rotation is cheap and touches only the rows that need touching. Gram–Schmidt survives mainly because it produces the columns of $Q$ one at a time, which is occasionally what one wants – and because it generalises to infinite dimensions, where reflections do not obviously help.

## §8.4 Linear Least Squares

We finish with the problem that motivated all of this. Let $A$ be $m \times n$ with $m > n$ and let $\mathbf{b} \in \mathbb{R}^m$. The system $A\mathbf{x} = \mathbf{b}$ has more equations than unknowns and generically has no solution at all; this is the situation of fitting $n$ parameters to $m$ measurements. The sensible thing to ask for instead is the $\mathbf{x}$ minimising the residual.

**Definition 8.11** (Least Squares Problem)

Given $A \in \mathbb{R}^{m\times n}$ and $\mathbf{b} \in \mathbb{R}^m$, the **linear least squares problem** is to find $\mathbf{x} \in \mathbb{R}^n$ minimising $\|A\mathbf{x} - \mathbf{b}\|_2$.

**Theorem 8.12** (Normal Equations)

A vector $\mathbf{x} \in \mathbb{R}^n$ solves the least squares problem if and only if

$$A^\top(A\mathbf{x} - \mathbf{b}) = \mathbf{0}, \qquad \text{equivalently} \qquad A^\top A\mathbf{x} = A^\top \mathbf{b}.$$

*Proof.* ($\Leftarrow$) Suppose $A^\top(A\mathbf{x} - \mathbf{b}) = \mathbf{0}$ and let $\mathbf{v} \in \mathbb{R}^n$ be arbitrary. Writing $\mathbf{v} = \mathbf{x} + (\mathbf{v} - \mathbf{x})$ and expanding,

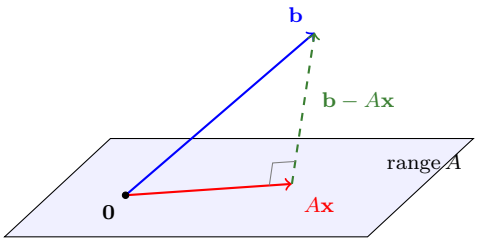
$$\begin{aligned} \|A\mathbf{v} - \mathbf{b}\|^2 &= \|(A\mathbf{x} - \mathbf{b}) + A(\mathbf{v} - \mathbf{x})\|^2 \\ &= \|A\mathbf{x} - \mathbf{b}\|^2 + 2(\mathbf{v} - \mathbf{x})^\top A^\top(A\mathbf{x} - \mathbf{b}) + \|A(\mathbf{v} - \mathbf{x})\|^2 \\ &= \|A\mathbf{x} - \mathbf{b}\|^2 + \|A(\mathbf{v} - \mathbf{x})\|^2 \;\geq\; \|A\mathbf{x} - \mathbf{b}\|^2 . \end{aligned}$$

($\Rightarrow$) Conversely, if $\mathbf{x}$ is a minimiser then for every $\mathbf{v}$ the scalar function $t \mapsto \|A(\mathbf{x} + t\mathbf{v}) - \mathbf{b}\|^2$ has a minimum at $t = 0$. Differentiating,

$$0 = \frac{d}{dt}\Big|_{t=0} \left( \|A\mathbf{x} - \mathbf{b}\|^2 + 2t\,\mathbf{v}^\top A^\top(A\mathbf{x} - \mathbf{b}) + t^2 \|A\mathbf{v}\|^2 \right) = 2\mathbf{v}^\top A^\top(A\mathbf{x} - \mathbf{b}),$$

and as $\mathbf{v}$ was arbitrary, $A^\top(A\mathbf{x} - \mathbf{b}) = \mathbf{0}$. □

The geometric content is that the residual must be orthogonal to the column space of $A$: the best approximation $A\mathbf{x}$ is the orthogonal projection of $\mathbf{b}$ onto $\operatorname{range} A$. This is Pythagoras again, exactly as in Section 2.




So one could simply solve the $n \times n$ system $A^\top A\mathbf{x} = A^\top \mathbf{b}$, say by Cholesky, since $A^\top A$ is symmetric and (if $A$ has full column rank) positive definite. This is what statisticians did for a century, and it is not wrong. But it has three defects that a numerical analyst notices immediately.

(i) Forming $A^\top A$ costs $\mathcal{O}(mn^2)$ operations and, more importantly, destroys any sparsity that $A$ possessed – a sparse $A$ can easily have a dense $A^\top A$.

(ii) The entries of $A^\top A$ are sums of products, so if the entries of $A$ vary greatly in magnitude, forming them loses accuracy through cancellation.

(iii) Most seriously, the conditioning of $A^\top A$ is the *square* of that of $A$. Squaring an already-large condition number is an excellent way to lose every digit one has.

The remedy is to use the QR factorisation, which lets us minimise the residual without ever forming $A^\top A$.

**Theorem 8.13** (Least Squares via QR)

Let $A$ be $m \times n$ with $m \geq n$ and linearly independent columns, and let $A = QR$ be a reduced QR factorisation, with $Q$ of size $m \times n$ having orthonormal columns and $R$ upper triangular and nonsingular. Then the least squares problem is solved by the unique solution of

$$R\mathbf{x} = Q^\top \mathbf{b}.$$

*Proof.* By Theorem 8.12, $\mathbf{x}$ is a solution if and only if $A^\top(A\mathbf{x}-\mathbf{b}) = \mathbf{0}$. Substituting $A = QR$ and using $Q^\top Q = I$,

$$A^\top(A\mathbf{x} - \mathbf{b}) = R^\top Q^\top(QR\mathbf{x} - \mathbf{b}) = R^\top\left(R\mathbf{x} - Q^\top\mathbf{b}\right).$$

As $R$ is nonsingular, so is $R^\top$, and the above vanishes precisely when $R\mathbf{x} = Q^\top\mathbf{b}$. That triangular system has a unique solution, found by back substitution. □

**Remark.** Using the full QR factorisation gives the same conclusion in a way that also exhibits the residual. If $A = QR$ with $Q$ orthogonal $m \times m$ and $R = \left(\begin{smallmatrix} R_1 \\ 0 \end{smallmatrix}\right)$ with $R_1$ upper triangular $n \times n$, then since $Q^\top$ preserves norms,

$$\|A\mathbf{x} - \mathbf{b}\|^2 = \left\|Q^\top A\mathbf{x} - Q^\top\mathbf{b}\right\|^2 = \|R_1\mathbf{x} - \mathbf{c}_1\|^2 + \|\mathbf{c}_2\|^2,$$

where $Q^\top\mathbf{b} = \left(\begin{smallmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{smallmatrix}\right)$ is split after $n$ entries. The second term does not involve $\mathbf{x}$ at all, so the minimum is attained by solving $R_1\mathbf{x} = \mathbf{c}_1$, and the minimum value of the residual is exactly $\|\mathbf{c}_2\|$. This is perhaps the cleanest way to see the whole result: an orthogonal change of coordinates turns an unsolvable system into a solvable one plus an irreducible remainder.

**Example 8.14** (Fitting a Straight Line)

Suppose we wish to fit $y = c_0 + c_1 t$ to the four data points $(0,1)$, $(1,3)$, $(2,4)$, $(3,4)$. Then

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 4 \end{pmatrix},$$

and the normal equations read

$$\begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} 12 \\ 23 \end{pmatrix},$$

with solution $c_1 = 1$, $c_0 = \frac{3}{2}$. So the best-fitting line is $y = \frac{3}{2} + t$, and the residual vector is $\left(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, -\frac{1}{2}\right)^\top$, whose components sum to zero and which is orthogonal to $(0,1,2,3)^\top$ – exactly the two normal equations, read backwards.

**Remark.** It is worth stepping back at the end. The least squares problem we have just solved is the same problem we solved in Section 2 with orthogonal polynomials, in a different costume: there the vector space was $C[a,b]$ or $\mathcal{P}_n$ and the orthogonal basis was $p_0, \ldots, p_n$; here it is $\mathbb{R}^m$ and the orthogonal basis consists of the columns of $Q$. In both cases the answer is an orthogonal projection, in both cases the good algorithm is the one that builds an orthogonal basis first, and in both cases the reason is the same – orthogonality decouples the problem and refuses to amplify error. If there is one idea to take away from this course, it is probably that one.